

The University of Faisalabad

Department of Computational Sciences

BSAI 6th Semester

Course Title: Data Mining

Course Code: DS-303

Name: Yumna Irfan

Reg # : 2023-bs-ai-021

LAB MANUAL

(2023-BS-AI-021)

Mr. Arham

Table of contents

Lab 1	03
Lab 2	19
Lab 3	27
Lab 4	29
Lab 5	34
Lab 6	37
Lab 7	40
Lab 8	43
Lab 9	46
Lab 10.....	49
Lab 11.....	54
Lab 12.....	58
Lab 13.....	62
Lab 14.....	67
Lab 15.....	72

LAB-1

1. Introduction to Data Mining
2. Software Introduction
3. Overview of datasets

CLO:

Understand data mining concepts, preprocessing, statistics, and trends

OBJECTIVE:

1. Understand the concept and applications of Data Mining
2. Use Google Colab as a development environment
3. Identify different types of datasets used in Data Mining

REQUIRED SOFTWARE / TOOLS

- Web Browser
- Google Account
- Google Colab
- Python (pre-installed in Colab)

Introduction to Data Mining

Data Mining is the process of discovering **useful patterns, trends, and knowledge** from large datasets using statistical, machine learning, and computational techniques.

Applications of Data Mining:

- Market basket analysis
- Fraud detection
- Healthcare diagnosis
- Social media analysis
- Recommendation systems

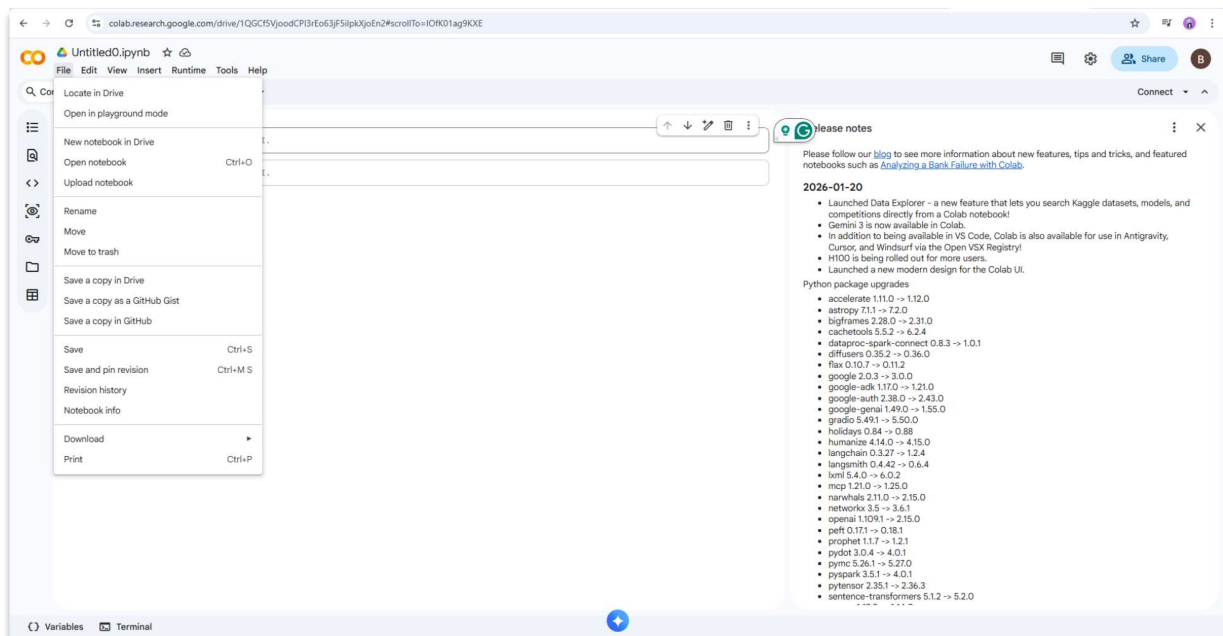
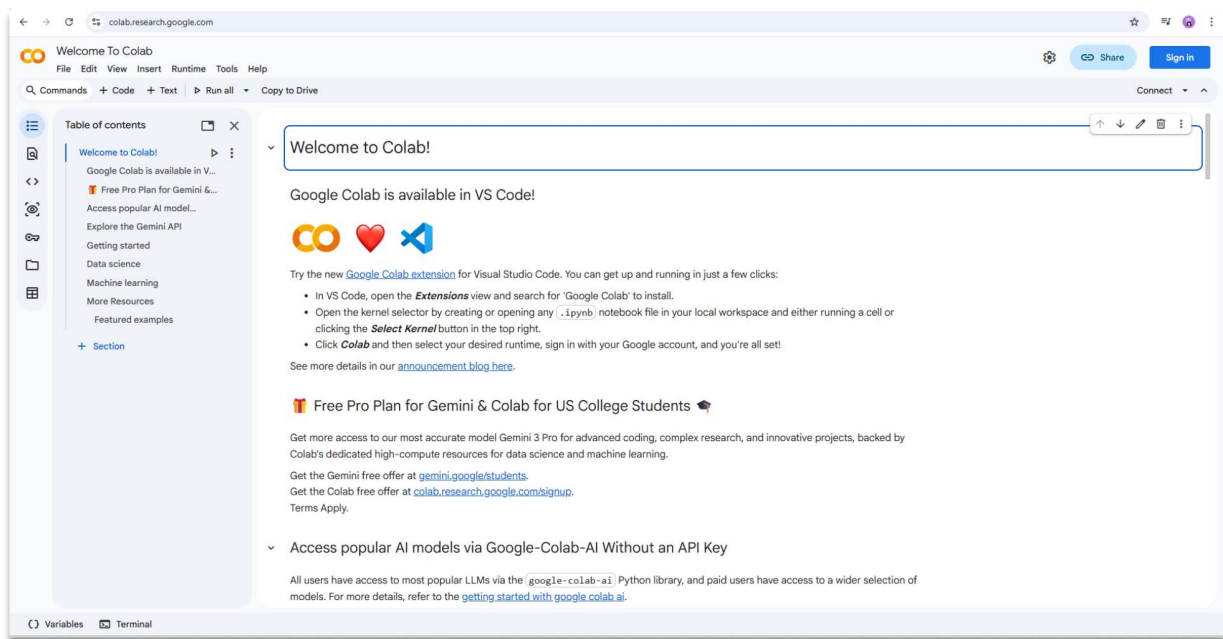
Software Introduction – Google Colab

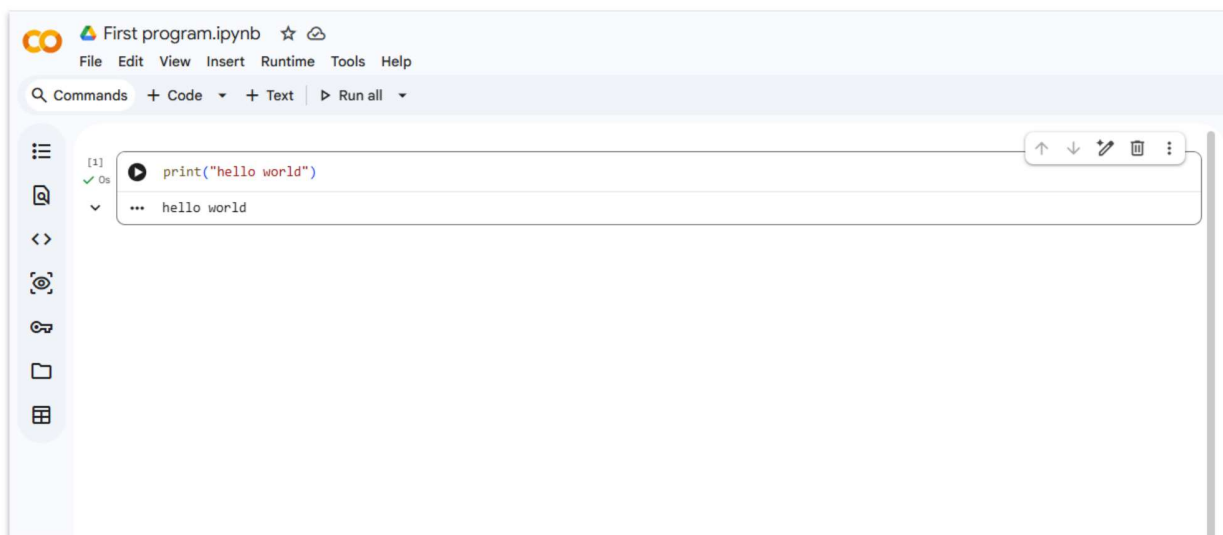
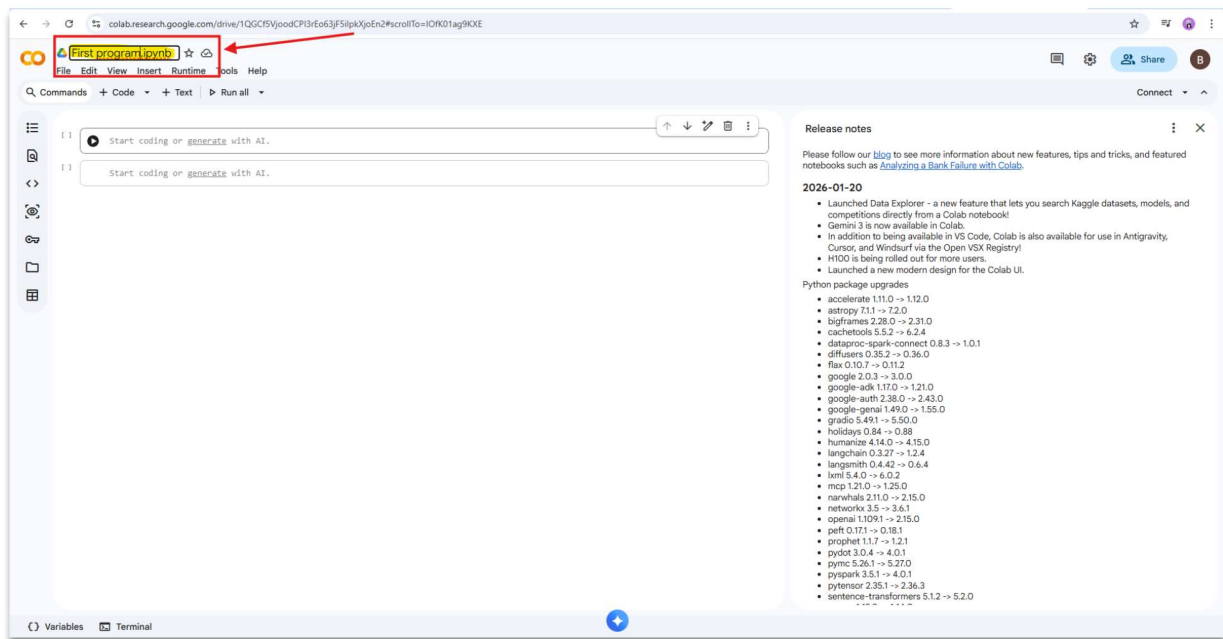
What is Google Colab?

Google Collaboratory (Colab) is a free cloud-based platform that allows users to write and execute Python code using a browser.

Features

- No installation required
- Free access to GPU/TPU
- Easy sharing and collaboration
- Pre-installed libraries (NumPy, Pandas, Matplotlib, etc.)





Overview of datasets

What is a Dataset in Data Mining?

A **dataset** is a structured collection of data used to discover patterns, relationships, or knowledge through data mining techniques.

In labs, datasets are usually stored as **tables** (rows = records, columns = attributes).

Common Types of Datasets

Structured Datasets

Data is organized in rows and columns.

Examples:

- Student performance data
- Sales transactions
- Bank customer records

Formats: CSV, Excel, SQL tables

Used for: Classification, clustering, regression

Transactional Datasets

Each record represents a transaction with a set of items.

Example:

```
T1 → {Milk, Bread}
T2 → {Milk, Diaper, Beer}
```

Used for:

- Association rule mining
- Market basket analysis
(Algorithms: Apriori, FP-Growth)

Time-Series Datasets

Data collected over time at regular intervals.

Examples:

- Stock prices
- Temperature readings
- Website traffic per day

Used for:

- Trend analysis
- Forecasting
- Sequential pattern mining

Spatial or Geographical Datasets

Data related to location or space.

Examples:

- GPS coordinates
- Maps
- Satellite images

Used for:

- Spatial clustering
- Location-based analysis

Text Datasets

Data in textual form.

Examples:

- Emails
- Reviews
- Tweets

Used for:

- Text mining
- Sentiment analysis
- Topic modeling

Web and Log Datasets

Data generated from web activity.

Examples:

- Clickstream data
- Server logs
- User browsing history

Used for:

- Web usage mining
- Recommendation systems

Attributes in a Dataset

1 Numerical Attributes

- Integer or real values
- Example: Age, Salary

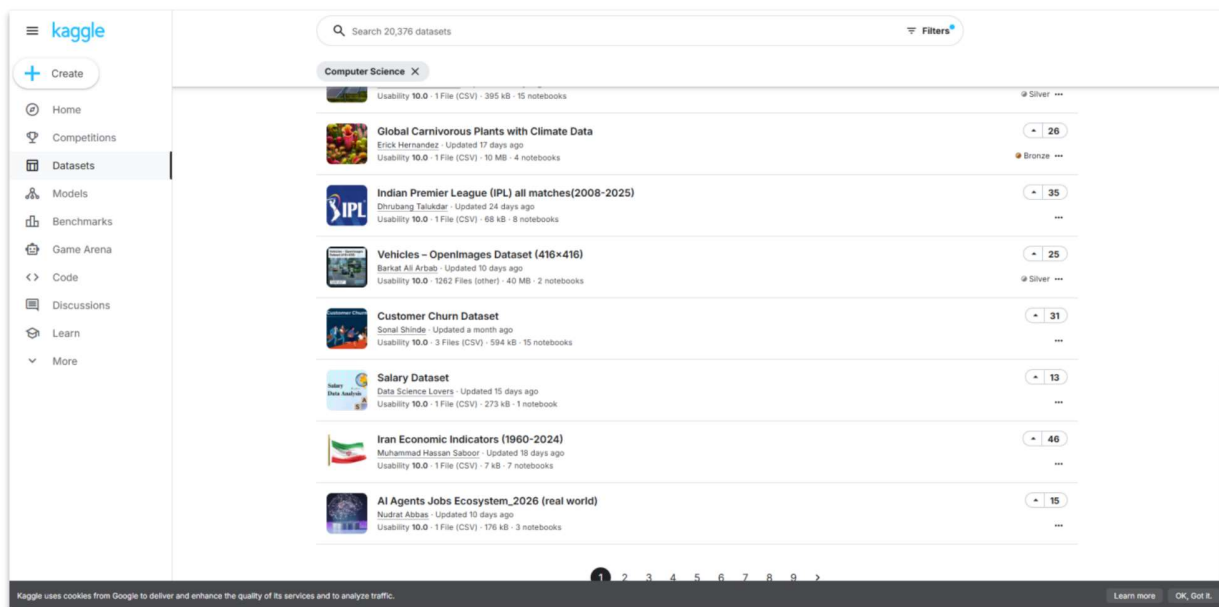
2 Categorical Attributes

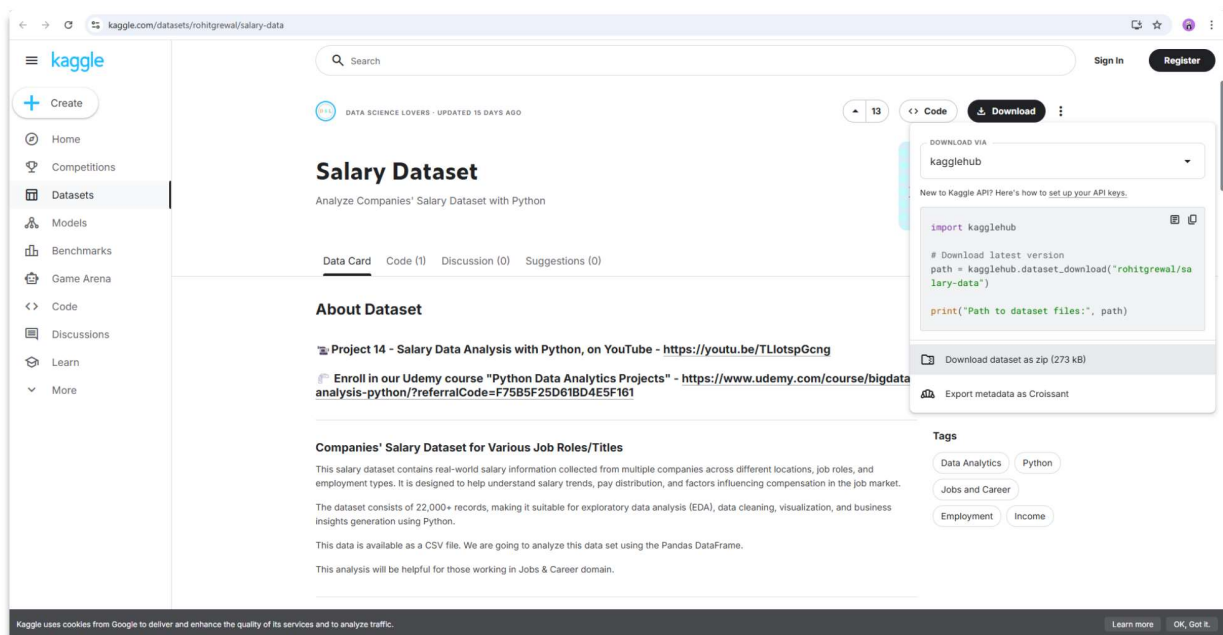
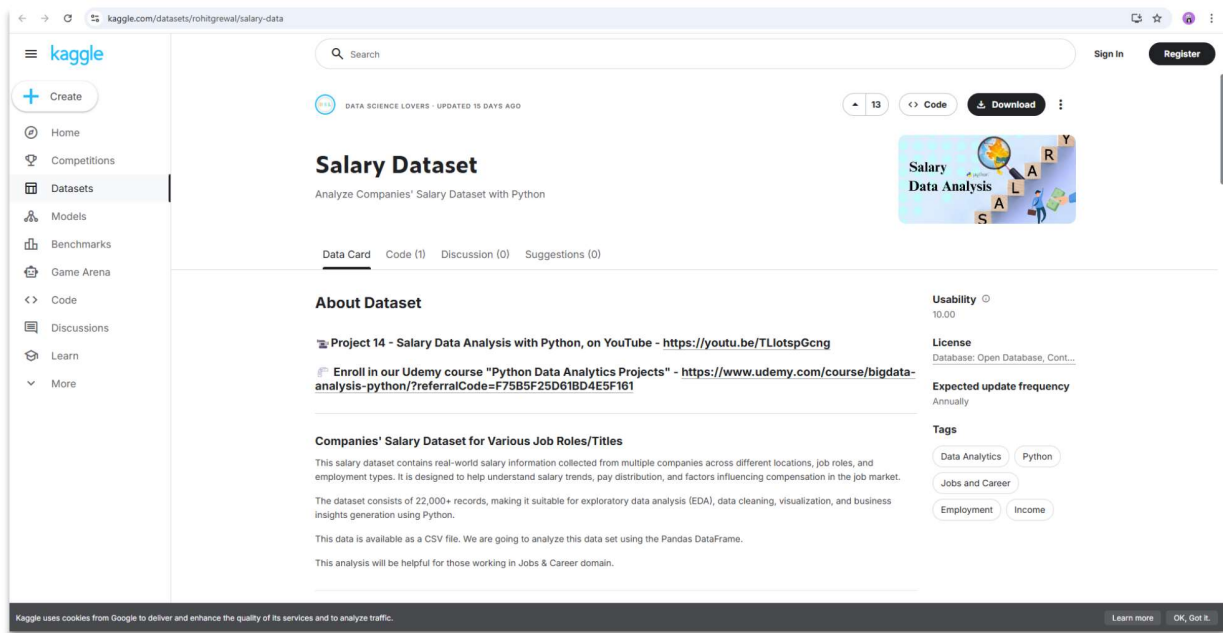
- Distinct categories
- Example: Gender, Department

3 Binary Attributes

- Two possible values
- Example: Yes/No, 0/1

How to download the dataset





TASK:

Write a Python program that does the following:

1. Create a list of integers entered by the user.
2. Using a **loop**, go through each number in the list.
3. For each number:
 - If the number is **even**, print "Even"
 - If the number is **odd**, print "Odd"

4. At the end, print:
 - The **total count of even numbers**
 - The **total count of odd numbers**

INPUT

Enter numbers separated by spaces: 3 10 7 8 12

OUTPUT

```
3 is Odd
10 is Even
7 is Odd
8 is Even
12 is Even

Total Even numbers: 3
Total Odd numbers: 2
```

Introduction to Knime

KNIME (Konstanz Information Miner) is an open-source data analytics, reporting, and integration platform. It is designed to help users build **data workflows visually**, without heavy coding.

What is KNIME Used For?

KNIME is commonly used for:

- Data analysis
- Machine learning
- Data cleaning & preprocessing
- Data visualization
- Predictive modeling
- Business intelligence

Task 1:

Working on Iris Dataset:

	A	B	C	D	E	F	G
1	Id	SepalLength	SepalWidth	PetalLength	PetalWidth	Species	
2	1	5.1	3.5	1.4	0.2	Iris-setosa	
3	2	4.9	3	1.4	0.2	Iris-setosa	
4	3	4.7	3.2	1.3	0.2	Iris-setosa	
5	4	4.6	3.1	1.5	0.2	Iris-setosa	
6	5	5	3.6	1.4	0.2	Iris-setosa	
7	6	5.4	3.9	1.7	0.4	Iris-setosa	
8	7	4.6	3.4	1.4	0.3	Iris-setosa	
9	8	5	3.4	1.5	0.2	Iris-setosa	
10	9	4.4	2.9	1.4	0.2	Iris-setosa	
11	10	4.9	3.1	1.5	0.1	Iris-setosa	
12	11	5.4	3.7	1.5	0.2	Iris-setosa	
13	12	4.8	3.4	1.6	0.2	Iris-setosa	
14	13	4.8	3	1.4	0.1	Iris-setosa	
15	14	4.3	3	1.1	0.1	Iris-setosa	
16	15	5.8	4	1.2	0.2	Iris-setosa	
17	16	5.7	4.4	1.5	0.4	Iris-setosa	
18	17	5.4	3.9	1.3	0.4	Iris-setosa	
19	18	5.1	3.5	1.4	0.3	Iris-setosa	
20	19	5.7	3.8	1.7	0.3	Iris-setosa	

Calculating Sum, Count, Min, Max, Average, Mode:

By using the following formulas we calculate all these:

=COUNT(A1:A151)

=SUM(A1:A151)

=AVERAGE(A1:A151)

=MODE(B2:B151)

=MIN(A1:A151)

=MAX(A1:A151)

11325	876.5	458.1	563.8	179.8
150	150	150	150	150
75.5	5.8433333	3.054	3.7586667	1.1986667
	5	3	1.5	0.2
1	4.3	2	1	0.1
150	7.9	4.4	6.9	2.5

CTRL + T

1	Id	SepalLen	SepalWid	PetalLen	PetalWid	Species
2	1	5.1	3.5	1.4	0.2	Iris-setosa
3	2	4.9	3	1.4	0.2	Iris-setosa
4	3	4.7	3.2	1.3	0.2	Iris-setosa
5	4	4.6	3.1	1.5	0.2	Iris-setosa
6	5	5	3.6	1.4	0.2	Iris-setosa
7	6	5.4	3.9	1.7	0.4	Iris-setosa
8	7	4.6	3.4	1.4	0.3	Iris-setosa
9	8	5	3.4	1.5	0.2	Iris-setosa
10	9	4.4	2.9	1.4	0.2	Iris-setosa

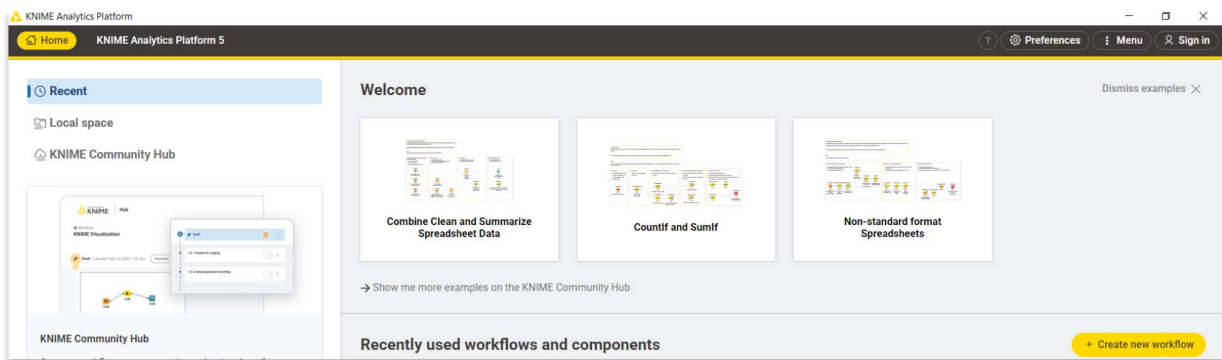
Format Painter | Paste | Table1 | fx | 1 | Orientation | Merge and Center | General | Conditional Formatting | Editing | Settings

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
1	id	Sepal.len	Sepal.Wid	Petal.len	Petal.Wid	Species								
2	1	5.1	3.5	1.4	0.2	Iris-setosa								
3	2	4.9	3	1.4	0.2	Iris-setosa								
4	3	4.7	3.2	1.3	0.2	Iris-setosa								
5	4	4.6	3.1	1.5	0.2	Iris-setosa								
6	5	5	3.6	1.4	0.2	Iris-setosa								
7	6	5.4	3.9	1.7	0.4	Iris-setosa								
8	7	4.6	3.4	1.4	0.3	Iris-setosa								
9	8	5	3.4	1.5	0.2	Iris-setosa								
10	9	4.4	2.9	1.4	0.2	Iris-setosa								
11	10	4.9	3.1	1.5	0.1	Iris-setosa								
12	11	5.4	3.7	1.5	0.2	Iris-setosa								
13	12	4.8	3.4	1.6	0.2	Iris-setosa								
14	13	4.8	3	1.4	0.1	Iris-setosa								
15	14	4.3	3	1.1	0.1	Iris-setosa								
16	15	5.8	4	1.2	0.2	Iris-setosa								
17	16	5.7	4.4	1.5	0.4	Iris-setosa								

Highlight Cells Rules | Top/Bottom Rules | Data Bars | Color Scales | Icon Sets | New Rule... | Clear Rules | Manage Rules...

1	Id	SepalLen	SepalWid	PetalLen	PetalWid	Species
2	1	5.1	3.5	1.4	0.2	Iris-setosa
3	2	4.9	3	1.4	0.2	Iris-setosa
4	3	4.7	3.2	1.3	0.2	Iris-setosa
5	4	4.6	3.1	1.5	0.2	Iris-setosa
6	5	5	3.6	1.4	0.2	Iris-setosa
7	6	5.4	3.9	1.7	0.4	Iris-setosa
8	7	4.6	3.4	1.4	0.3	Iris-setosa
9	8	5	3.4	1.5	0.2	Iris-setosa
10	9	4.4	2.9	1.4	0.2	Iris-setosa

Working on knime



Click on create new workflow:

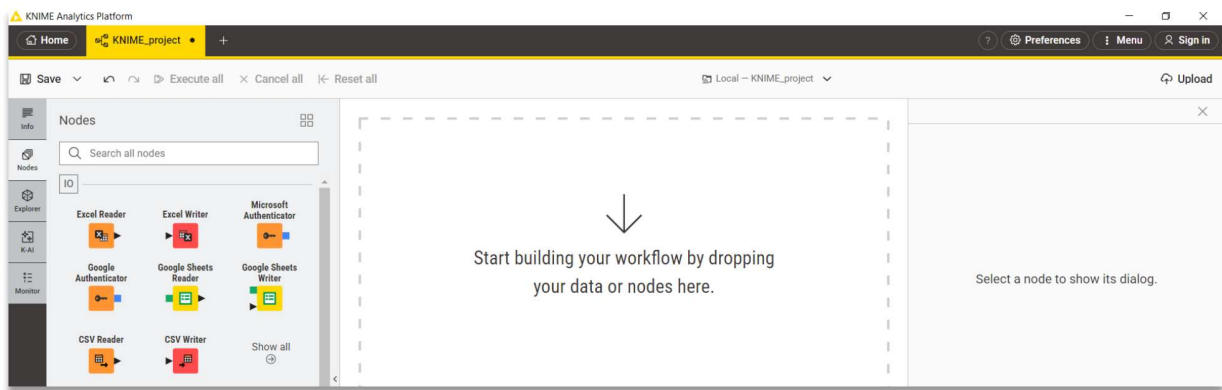
Create a new workflow

Workflow name

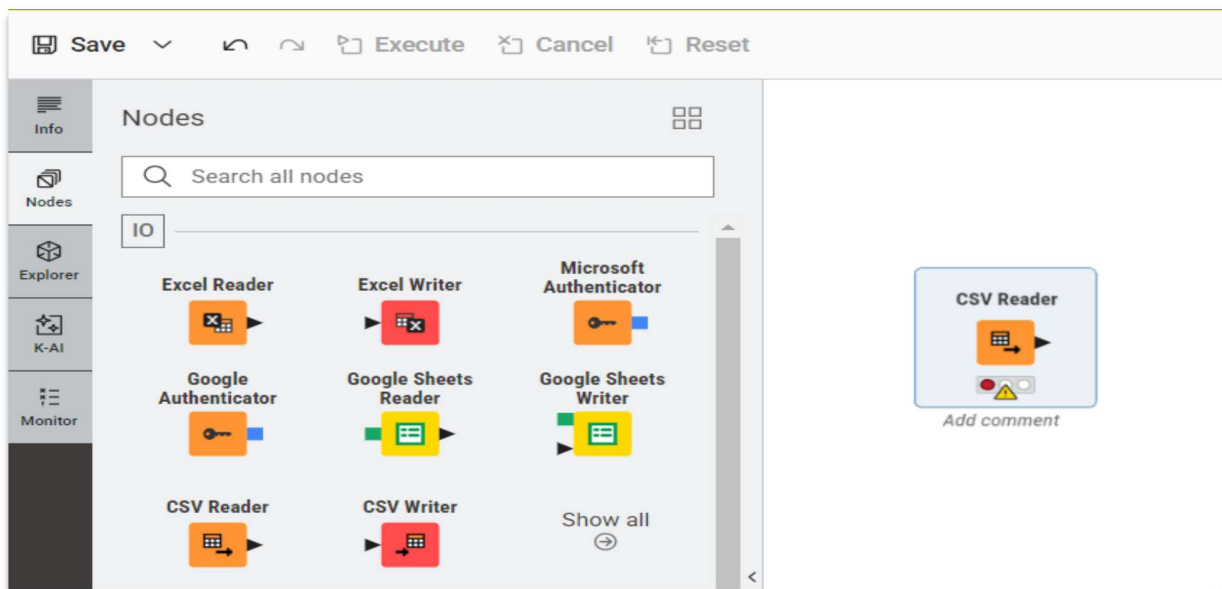
KNIME_Project

Cancel

Create



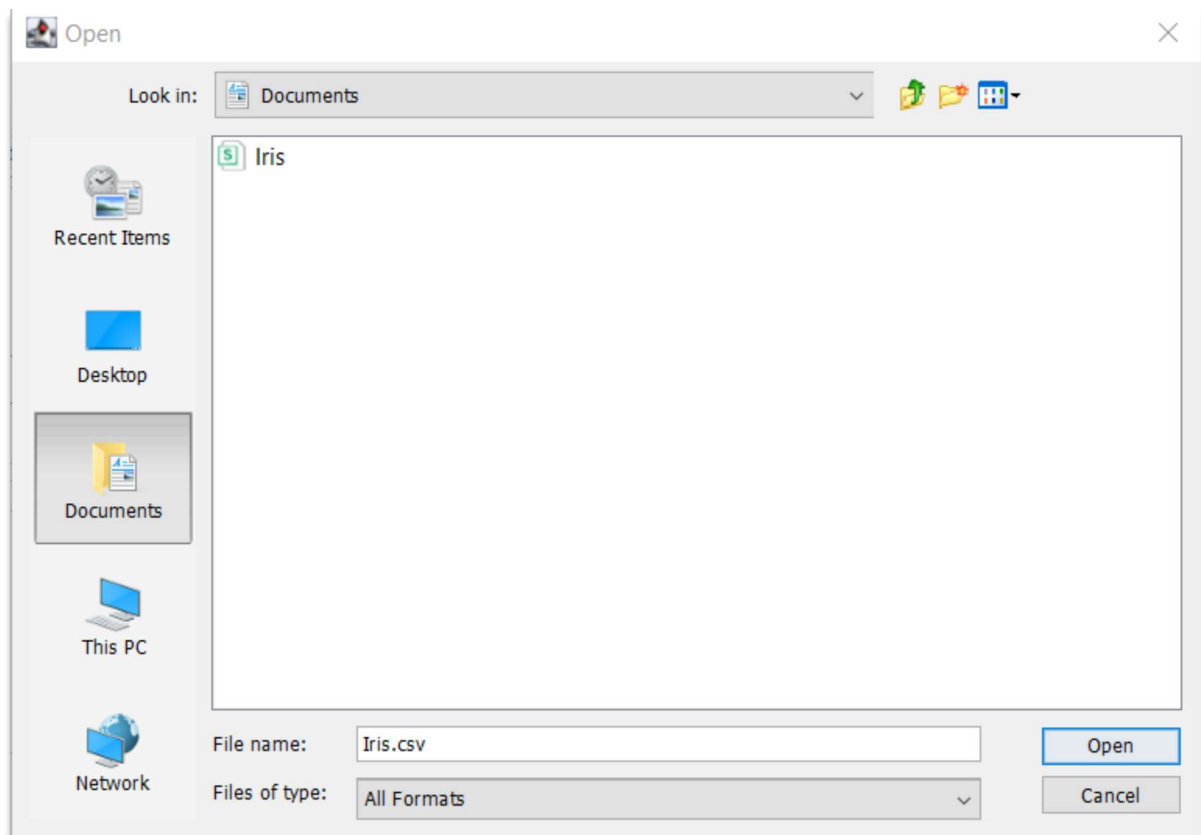
In Nodes section choose CSV reader:



Browse the file u want to upload:



Select the file and click open:



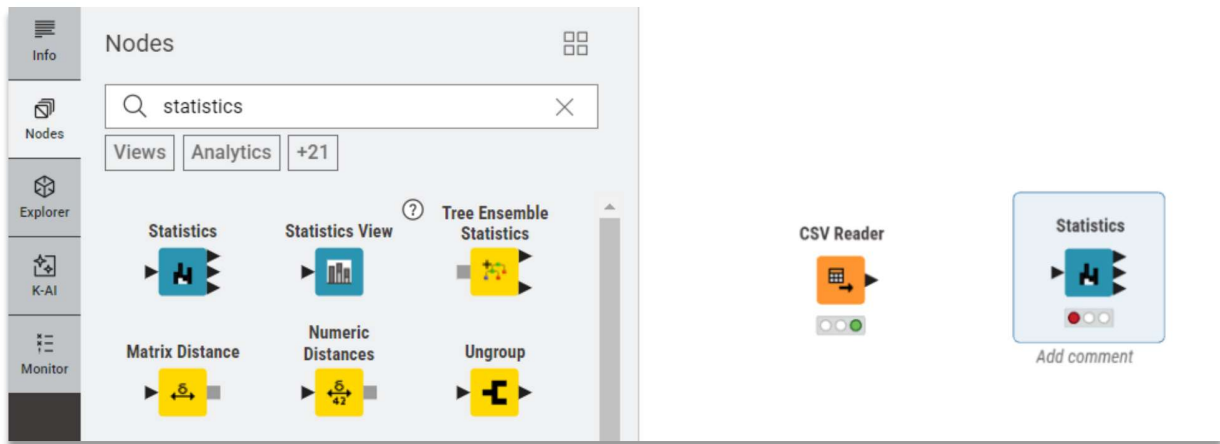
After clicking execute the CSV file will be uploaded in knime environment:

► 1: File Table Flow Variables

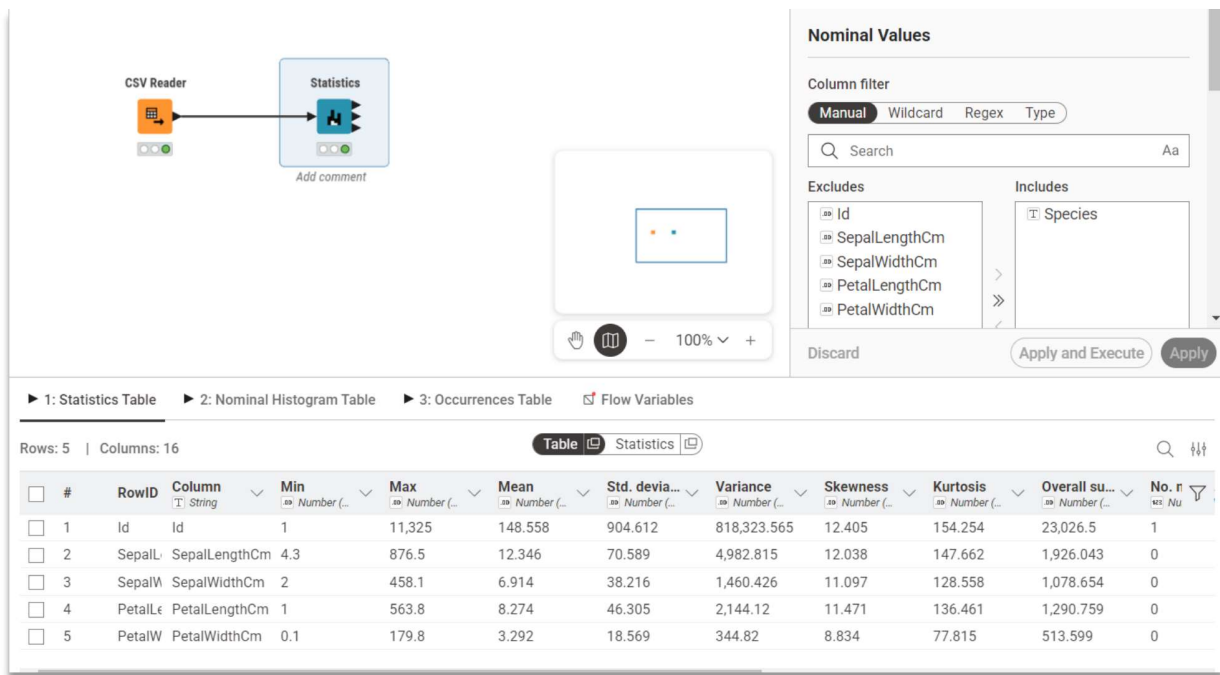
Rows: 156 | Columns: 6 Table

☐	#	RowID	Id Number (Float)	SepalLengthCm Number (Float)	SepalWidthCm Number (Float)	PetalLengthCm Number (Float)	PetalWidthCm Number (Float)	Species String
☐	1	Row0	1	5.1	3.5	1.4	0.2	Iris-setosa
☐	2	Row1	2	4.9	3	1.4	0.2	Iris-setosa
☐	3	Row2	3	4.7	3.2	1.3	0.2	Iris-setosa
☐	4	Row3	4	4.6	3.1	1.5	0.2	Iris-setosa
☐	5	Row4	5	5	3.6	1.4	0.2	Iris-setosa
☐	6	Row5	6	5.4	3.9	1.7	0.4	Iris-setosa

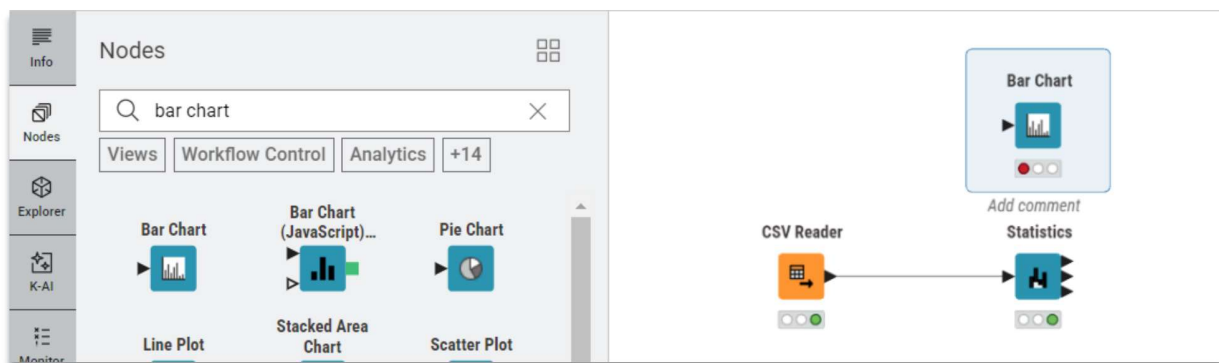
Now use statistic node:



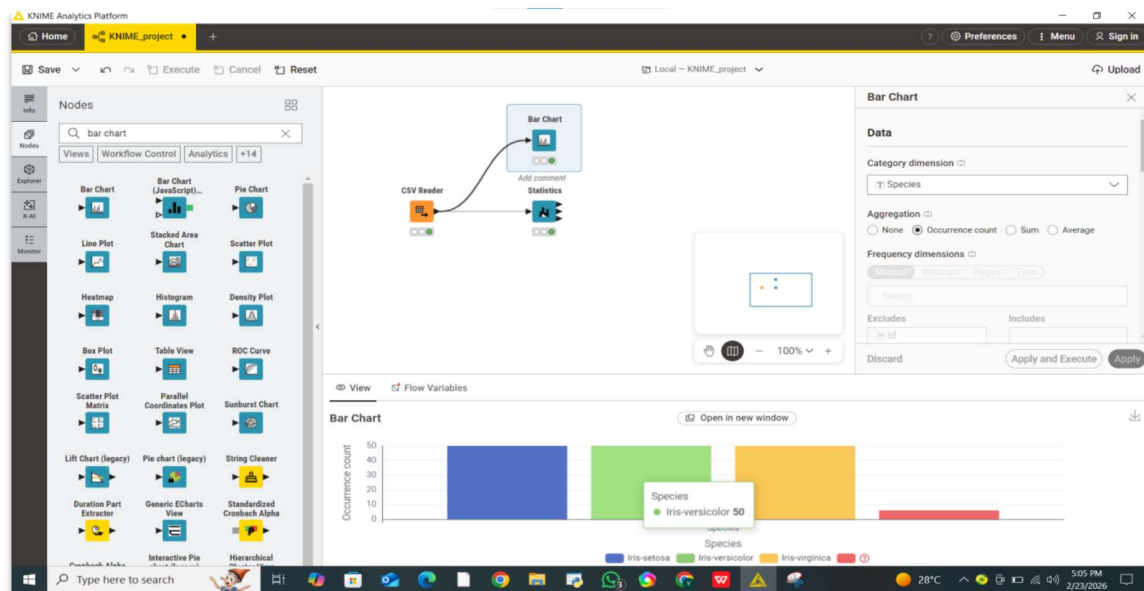
By joining it to the CSV node it will automatically display the mean, min, max, std dev etc.



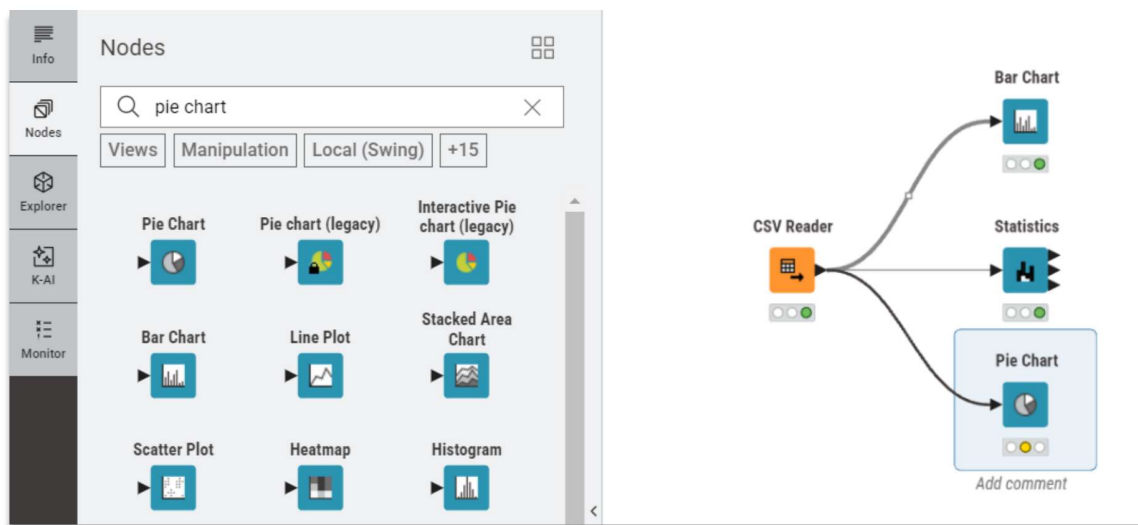
Now add barchart node:



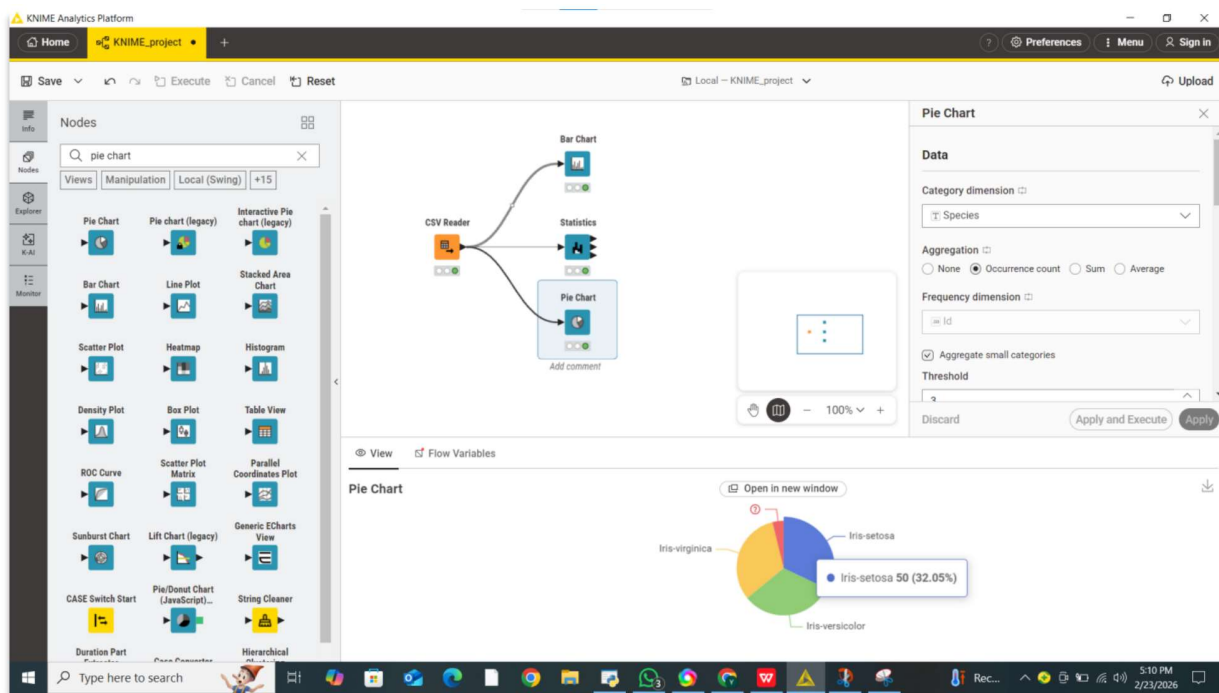
It will display the count of each specie:



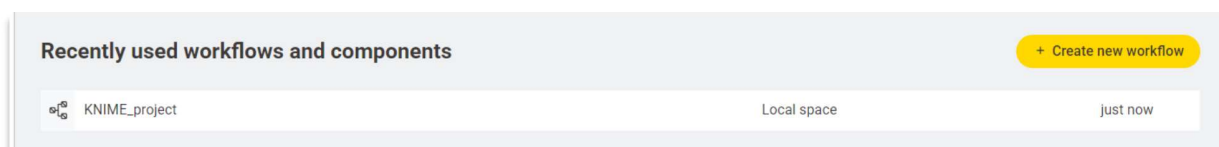
Now add the pie chart node:



It will display the percentage of each specie in pie chart:



Now save the workflow:



LAB-2

CRISP-DM Process And Data Types

CRISP-DM Process

CRISP-DM (Cross-Industry Standard Process for Data Mining) is a well-established framework used to guide data mining projects. It provides a clear, step-by-step structure to ensure an organized workflow. The process consists of six key stages: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment.

1. **Business Understanding** focuses on identifying the problem and setting clear objectives.
2. **Data Understanding** involves gathering data and analyzing its characteristics.
3. **Data Preparation** includes cleaning the dataset, transforming variables, and organizing it for analysis.
4. **Modeling** is where appropriate algorithms are selected and applied to the data.
5. **Evaluation** checks whether the model achieves the intended goals and performs effectively.
6. **Deployment** is the final step, where the model is put into practical use.

For instance, in a sales forecasting project, CRISP-DM ensures a systematic process from defining the problem to implementing the final solution.

Types of Data

Data can be categorized based on its characteristics:

- **Numerical Data** includes measurable values and can be either discrete (e.g., number of items) or continuous (e.g., height or income).
- **Categorical Data** represents groups or labels and includes:
 - *Nominal data* (e.g., colors, names)
 - *Ordinal data* (e.g., rankings or levels)
- **Binary Data** consists of only two possible values, such as 0 and 1.

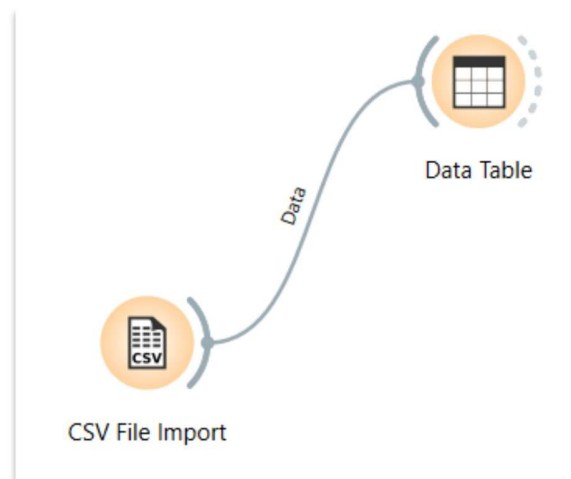
Recognizing the type of data is essential because different machine learning algorithms require specific input formats. For example, decision tree models can easily work with categorical variables, while linear regression models typically require numerical inputs.

Real-World Applications

Data mining plays a significant role in many industries:

- In **healthcare**, it is used for predicting diseases and assisting in diagnosis.

- In **retail**, it supports recommendation systems and analyzes customer purchasing patterns.
- In **finance**, it helps identify fraudulent activities and manage financial risks.
- On **social media platforms**, it is applied in sentiment analysis and personalized advertising.



Data Table - Orange

Info
500 instances (no missing data)
3 features
No target variable.
No meta attributes.

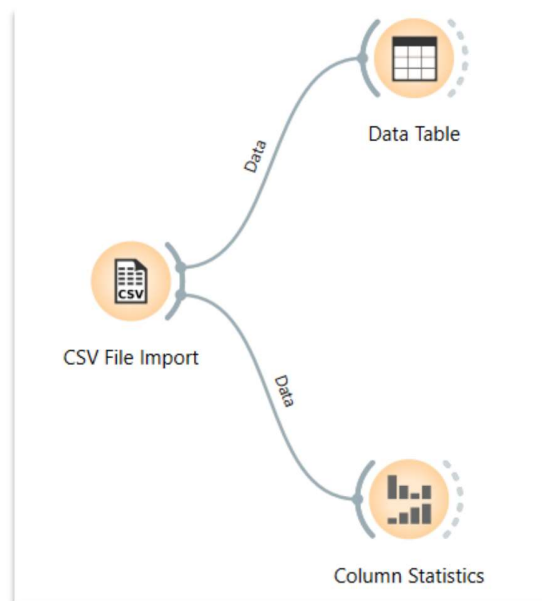
Variables
☒ Show variable labels (if present)
☐ Visualize numeric values
☒ Color by instance classes

Selection
Clear Selection
☒ Select full rows

Restore Original Order
☒ Send Automatically

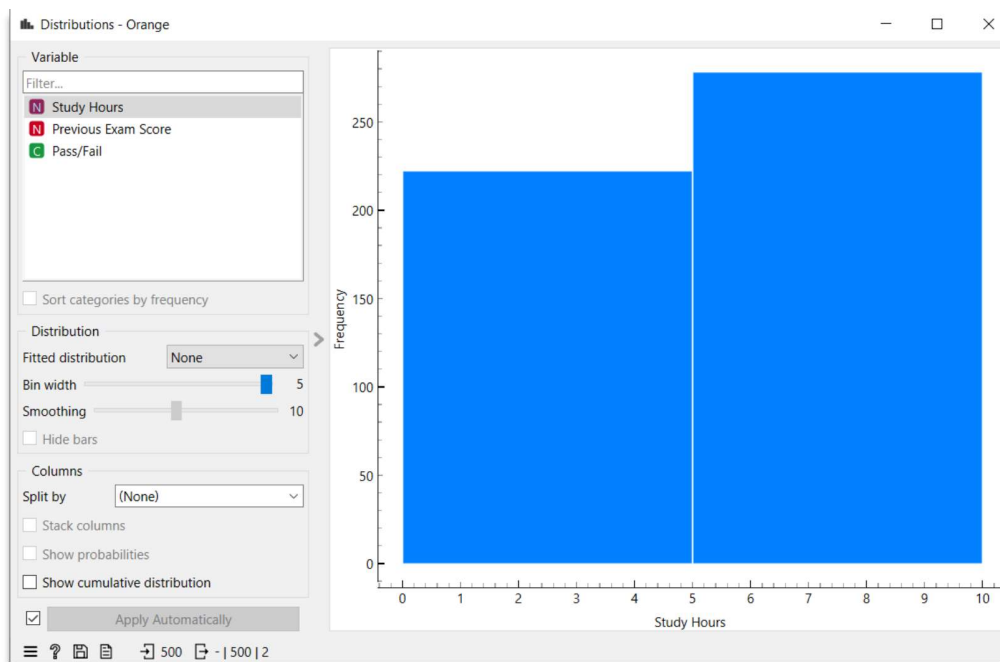
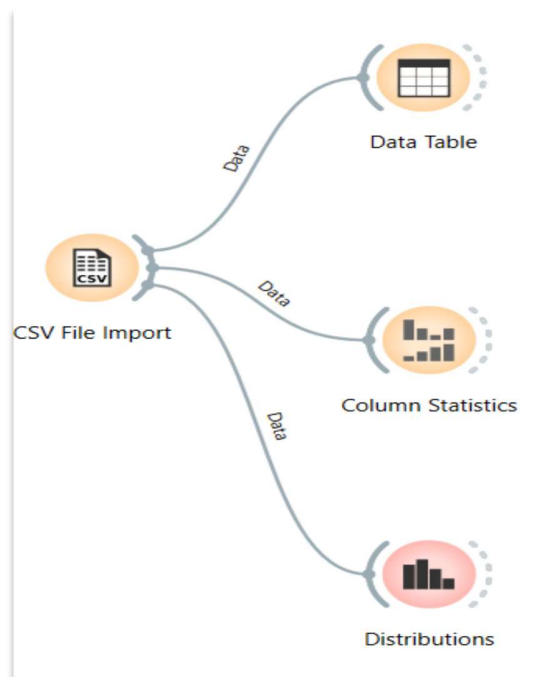
	Study Hours	revious Exam Scor	Pass/Fail
1	4.37086	81.8897	0
2	9.55643	72.1658	1
3	7.58795	58.5717	0
4	6.38793	88.8277	1
5	2.40417	81.0839	0
6	2.40395	49.757	0
7	1.52275	94.6556	0
8	8.79559	89.3522	1
9	6.41004	96.988	1
10	7.37265	83.5432	1
11	1.18526	76.8049	0
12	9.72919	65.0946	1
13	8.49198	95.9637	1
14	2.91105	91.9638	0
15	2.63642	42.7131	0
16	2.65064	41.582	0
17	3.73818	62.5878	0
18	5.72281	88.6332	1
19	4.88751	99.2366	0
20	3.62106	49.025	0
21	6.50668	75.6478	1
22	2.25544	62.8535	0

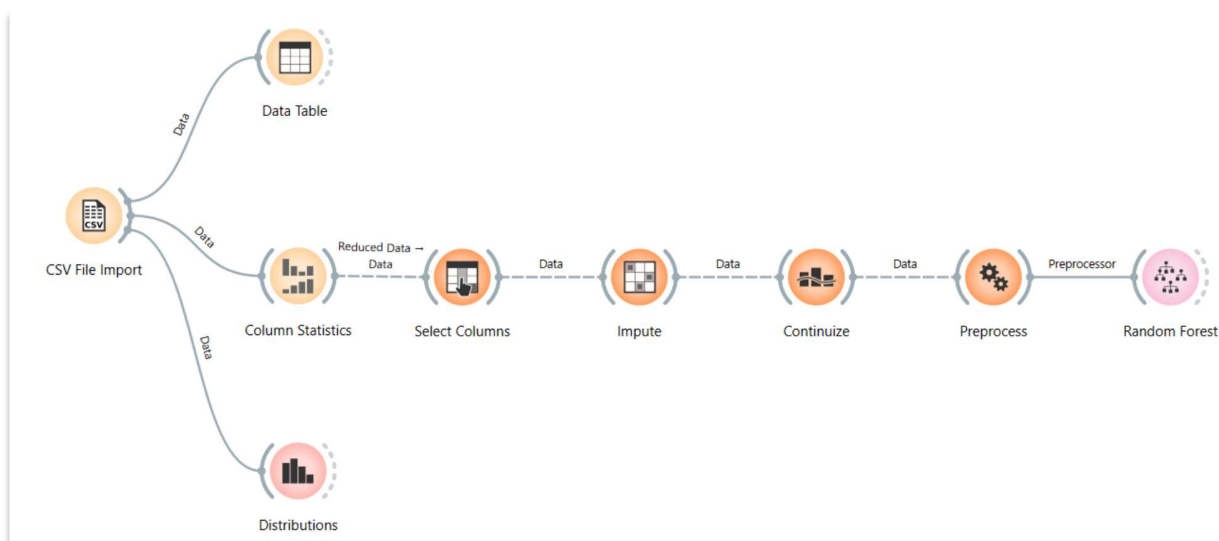
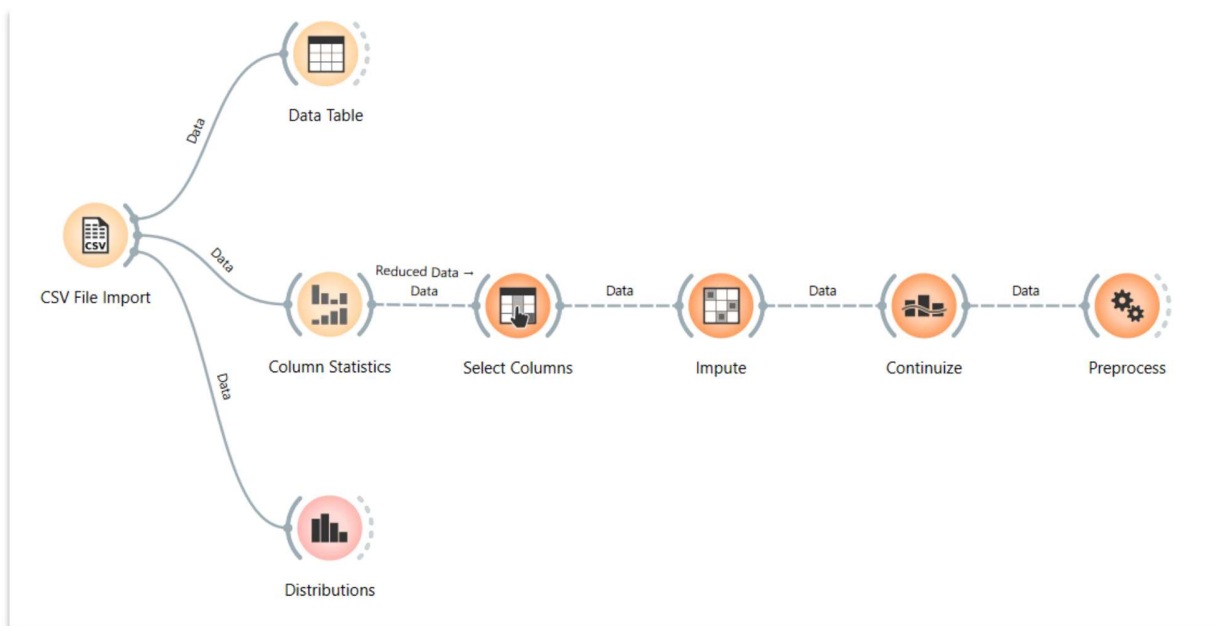
500 500 | 500

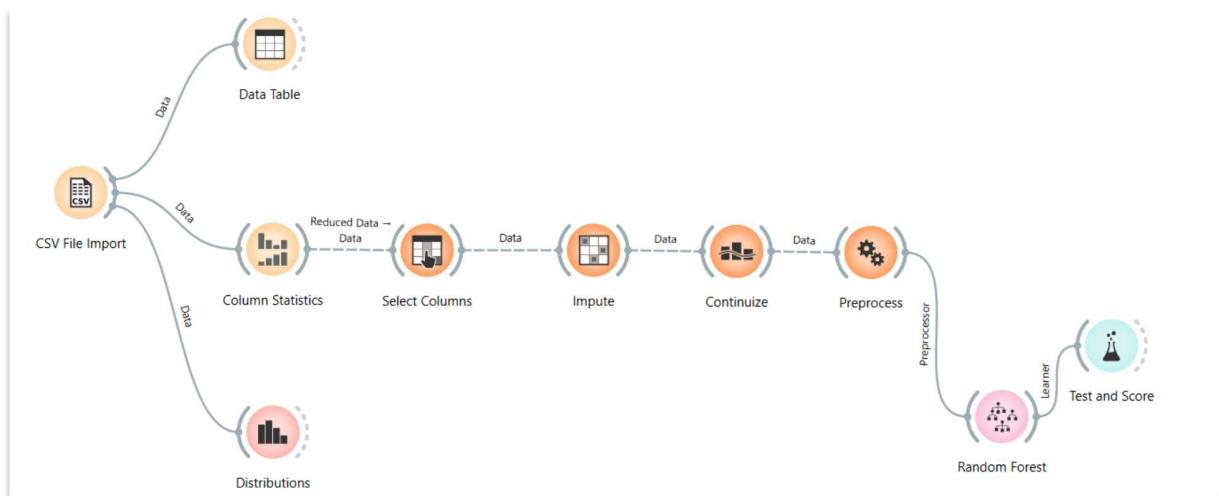


Column Statistics - Orange

	Column	Distribution	Mean	Mode	Median	Dispersion	Min.	Max.	Missing
N	Study Hours		5.48706	1.04555	5.61847	0.489426	1.04555	9.93668	0 (0 %)
N	Previous Exam Score		68.9171	40.2779	68.3093	0.248305	40.2779	99.9831	0 (0 %)
C	Pass/Fail			0		0.658			0 (0 %)





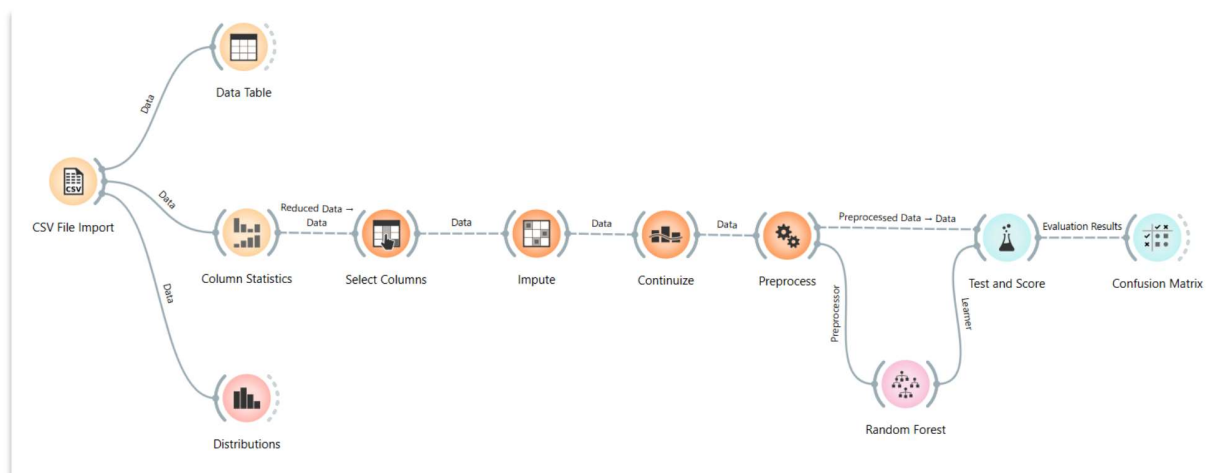


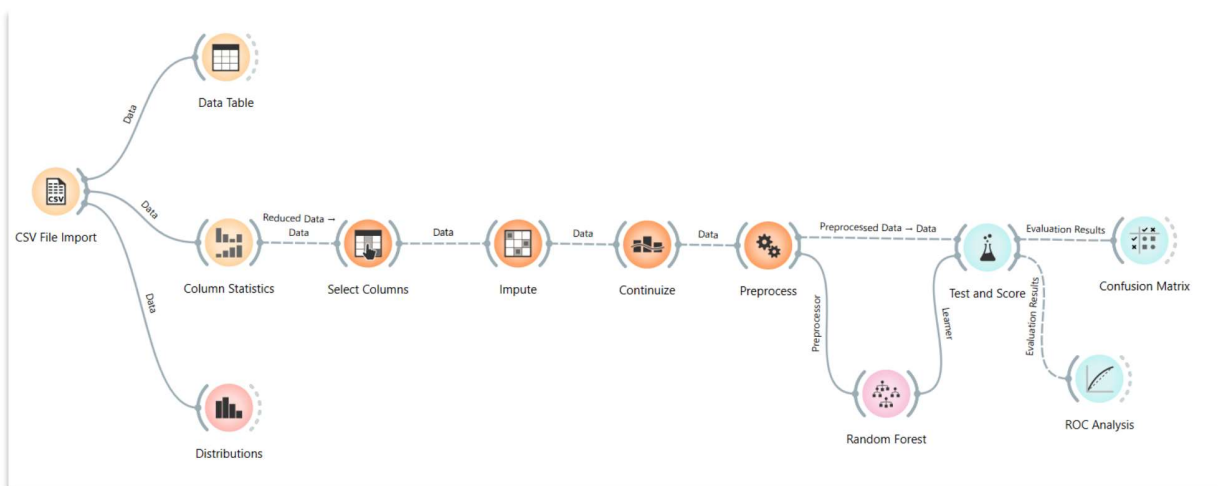
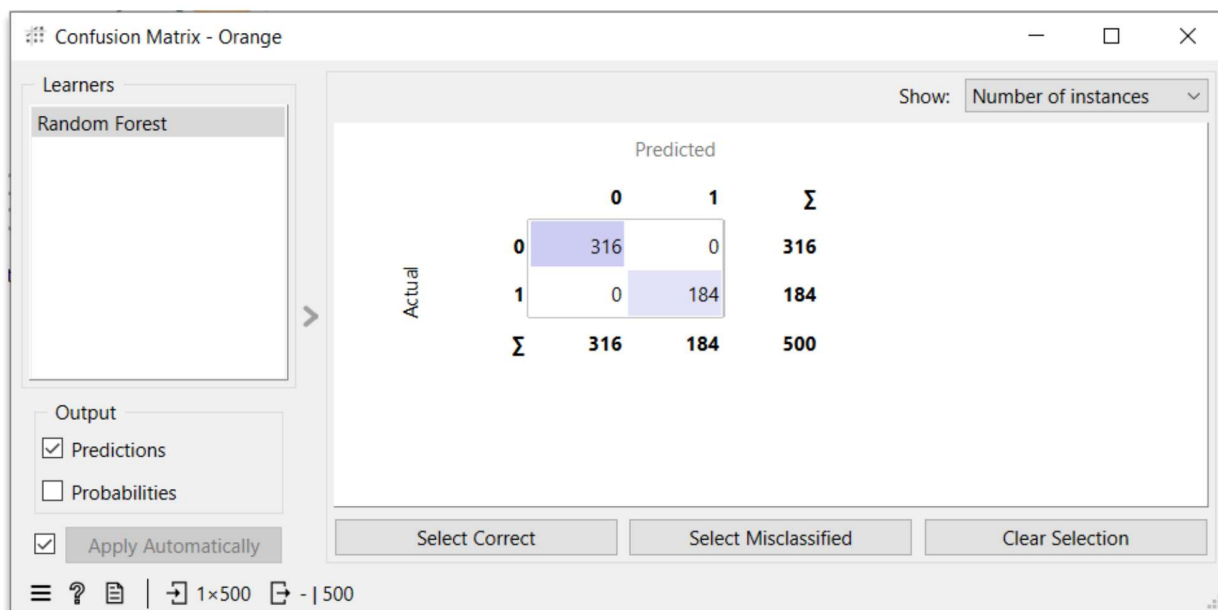
Test and Score - Orange

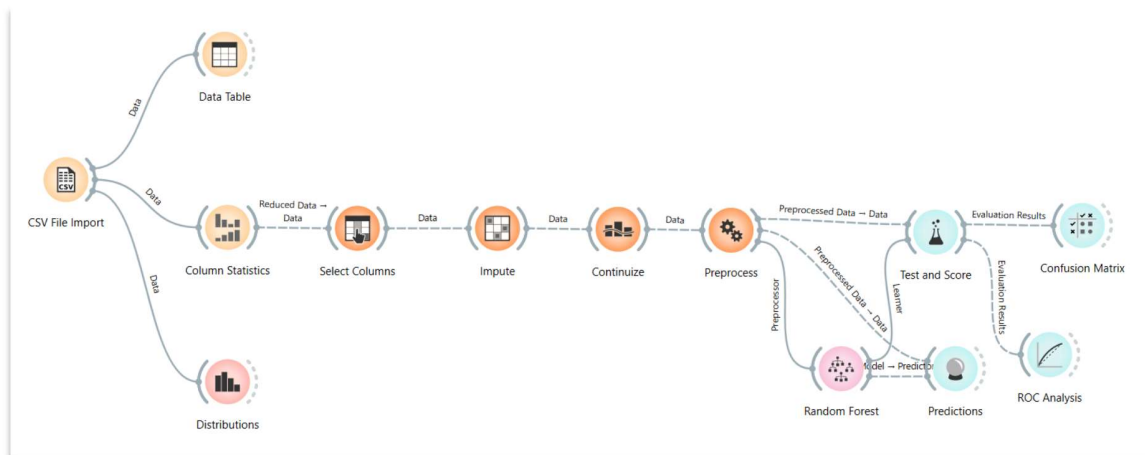
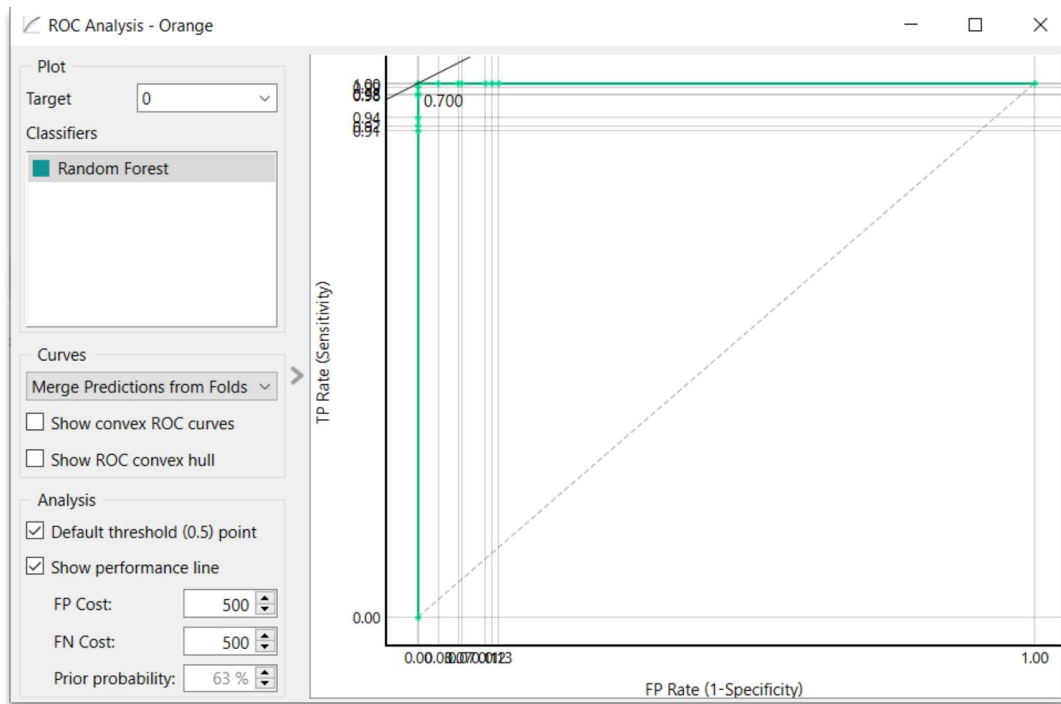
☒ Cross validation
 Number of folds: 5
☒ Stratified
☐ Cross validation by feature
☐ Random sampling
 Repeat train/test: 10
 Training set size: 66 %

Evaluation results for target (None, show average over classes)

Model	AUC	CA	F1	Prec	Recall	MCC
Random Forest	1.000	1.000	1.000	1.000	1.000	1.000







Predictions - Orange

Show probabilities for: Classes in data ☒ Show classification errors [Restore Original Order](#)

	Pass/Fail	Study Hours	revious Exam Scoi
0	4.37086	81.8897	
1	9.55643	72.1658	
0	7.58795	58.5717	
1	6.38793	88.8277	
0	2.40417	81.0839	

☒ Show performance scores Target class: (Average over classes)

500 | 500 | 500 | -

LAB-3

Data Cleaning Techniques

Handling Missing Values

Missing data appears when certain values are not recorded in a dataset, which can negatively impact analysis and model accuracy. To address this issue, simple methods such as removing records with missing entries or replacing them with statistical measures like the mean, median, or mode are commonly used. More advanced approaches include techniques like interpolation or using machine learning models to estimate the missing values. For instance, if the age field is missing for some entries, it can be filled using the average age calculated from the available data.

Removing Duplicates

Duplicate entries occur when identical records are present more than once in a dataset. These repetitions can distort analysis and produce misleading results. Therefore, it is important to detect and eliminate duplicates during the data preprocessing stage. For example, if the same customer information is recorded multiple times, the extra copies should be removed to ensure data accuracy and consistency.

Handling Noise Using Python

Noise in data refers to random errors or irregular variations that can reduce the quality of analysis. It can be minimized using methods such as smoothing, binning, regression techniques, or filtering. Python tools like Pandas and NumPy offer useful functions to clean and refine noisy datasets. For example, extreme values (outliers) in salary data can be identified and removed to achieve more reliable analytical results.

Data Cleaning Script for Healthcare Dataset

```
[1]: import pandas as pd
import numpy as np

Step 1: Load dataset

[2]: file_path = "dataset 3.csv"
df = pd.read_csv(file_path)

print("Initial Data Overview:")
df.info()

Initial Data Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 55500 entries, 0 to 55499
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Name                   55500 non-null  object  
1   Age                    55500 non-null  int64   
2   Gender                 55500 non-null  object  
3   Blood Type             55500 non-null  object  
4   Medical Condition       55500 non-null  object  
5   Date of Admission       55500 non-null  object  
6   Doctor                 55500 non-null  object  
7   Hospital               55500 non-null  object  
8   Insurance Provider      55500 non-null  object  
9   Billing Amount          55500 non-null  float64  
10  Room Number            55500 non-null  int64   
11  Admission Type          55500 non-null  object  
12  Discharge Date          55500 non-null  object  
13  Medication              55500 non-null  object  
14  Test Results            55500 non-null  object  
dtypes: float64(1), int64(2), object(12)
memory usage: 6.4+ MB
```

```
[3]: df.head()
```

```
[3]:
```

	Name	Age	Gender	Blood Type	Medical Condition	Date of Admission	Doctor	Hospital	Insurance Provider	Billing Amount	Room Number	Admission Type	Discharge Date	Medication	Test Results
0	Bobby JacksOn	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.281306	328	Urgent	2024-02-02	Paracetamol	Normal
1	LesLie TErRy	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.327287	265	Emergency	2019-08-26	Ibuprofen	Inconclusive
2	DaNnY sMitH	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.096079	205	Emergency	2022-10-07	Aspirin	Normal
3	andrEw waTtS	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.782410	450	Elective	2020-12-18	Ibuprofen	Abnormal
4	adriENNE bEll	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.317814	458	Urgent	2022-10-09	Penicillin	Abnormal

Step 2: Handle missing values

```
[4]: categorical_cols = df.select_dtypes(include=['object', 'category']).columns
```

Step 3: Remove duplicates

```
[5]: initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
duplicates_removed = initial_rows - df.shape[0]
print(f"Duplicates removed: {duplicates_removed}")

Duplicates removed: 534
```

Step 4: Noise filtering / Outlier removal (example using Z-score)

```
[6]: numeric_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

Step 5: Summary of cleaning

```
[7]: missing_after = df.isnull().sum().sum()
print(f"Total missing values after cleaning: {missing_after}")

Total missing values after cleaning: 0
```

Step 6: Save cleaned data

```
[8]: cleaned_file_path = "healthcare_data_cleaned.csv"
df.to_csv(cleaned_file_path, index=False)
print(f"Cleaned dataset saved to {cleaned_file_path}")

Cleaned dataset saved to healthcare_data_cleaned.csv
```


LAB-4

Data Preprocessing

Normalization

Normalization is a technique used to rescale data so that all values fall within a specific range, typically between 0 and 1. This is especially helpful when different features have varying scales, as it ensures consistency across the dataset.

The formula used is:

$$(X - \text{Min}) / (\text{Max} - \text{Min})$$

For example, features like salary and age can be normalized so they are comparable within the same range.

Standardization

Standardization adjusts data so that it has a mean of 0 and a standard deviation of 1. This transformation is useful for many machine learning algorithms that assume normally distributed data.

The formula is:

$$(X - \text{Mean}) / \text{Standard Deviation}$$

It is widely applied in algorithms such as Support Vector Machines (SVM) and K-Nearest Neighbors (KNN).

Discretization

Discretization is the process of converting continuous variables into distinct categories or intervals. This method simplifies data and can make patterns easier to identify.

For instance, age values can be grouped into categories like young, adult, and elderly. This can also enhance the performance of certain models.

Summary Statistics

Summary statistics provide a quick overview of the key characteristics of a dataset. Common measures include mean, median, mode, variance, and standard deviation.

These statistics help in understanding the overall distribution, central tendency, and variability within the data.

Python Basics

```
[1]: import pandas as pd

class DataProcessor:
    def __init__(self, file_path):
        self.file_path = file_path
        self.data = None

    def load_data(self):
        self.data = pd.read_csv(self.file_path)
        return self.data

    def show_info(self):
        info = {
            "Rows": self.data.shape[0],
            "Columns": self.data.shape[1],
            "Column Names": list(self.data.columns)
        }
        return info

    def handle_missing_values(self):
        self.data = self.data.fillna(self.data.mean(numeric_only=True))
        return self.data

    def remove_duplicates(self):
        self.data = self.data.drop_duplicates()
        return self.data

    def normalize_column(self, column_name):
        self.data[column_name] = (
            self.data[column_name] - self.data[column_name].min()
        ) / (
            self.data[column_name].max() - self.data[column_name].min()
        )
        return self.data

    def get_summary(self):
        return self.data.describe()
```

Lists

```
[2]: selected_columns = ['Age', 'Income', 'LoanAmount']
```

Dictionary

```
[3]: column_info = {
    "numeric": ['Age', 'Income', 'LoanAmount'],
    "categorical": ['Gender', 'Education']
}

print("Selected Columns:", selected_columns)
print("Column Info:", column_info)

Selected Columns: ['Age', 'Income', 'LoanAmount']
Column Info: {'numeric': ['Age', 'Income', 'LoanAmount'], 'categorical': ['Gender', 'Education']}
```

Load dataset

```
[4]: processor = DataProcessor("dataset 4.csv")
df = processor.load_data()
```

Display dataset info

```
[5]: print("\nDataset Info:")
print(processor.show_info())

Dataset Info:
{'Rows': 367, 'Columns': 12, 'Column Names': ['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Loan_Amount_Term', 'Credit_History', 'Property_Area']}
```

Preprocessing

```
[6]: processor.handle_missing_values()
processor.remove_duplicates()
```

```
[6]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.000000	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.000000	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.000000	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	0.825444	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.000000	
...	...	...	...	...	...	...	...	...	...	...	...	...
362	LP002971	Male	Yes	3+	Not Graduate	Yes	4009	1777	113.0	360.0	1.000000	
363	LP002975	Male	Yes	0	Graduate	No	4158	709	115.0	360.0	1.000000	
364	LP002980	Male	No	0	Graduate	No	3250	1993	126.0	360.0	0.825444	S
365	LP002986	Male	Yes	0	Graduate	No	5000	2393	158.0	360.0	1.000000	
366	LP002989	Male	No	0	Graduate	Yes	9200	0	98.0	180.0	1.000000	

367 rows × 12 columns



Normalize one column

```
[7]: processor.normalize_column("LoanAmount")
```

```
[7]:
```

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Prop
0	LP001015	Male	Yes	0	Graduate	No	5720	0	0.157088	360.0	1.000000	
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	0.187739	360.0	1.000000	
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	0.344828	360.0	1.000000	
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	0.137931	360.0	0.825444	
4	LP001051	Male	No	0	Not Graduate	No	3276	0	0.095785	360.0	1.000000	
...	...	...	...	...	...	...	...	...	...	...	...	...
362	LP002971	Male	Yes	3+	Not Graduate	Yes	4009	1777	0.162835	360.0	1.000000	
363	LP002975	Male	Yes	0	Graduate	No	4158	709	0.166667	360.0	1.000000	
364	LP002980	Male	No	0	Graduate	No	3250	1993	0.187739	360.0	0.825444	
365	LP002986	Male	Yes	0	Graduate	No	5000	2393	0.249042	360.0	1.000000	
366	LP002989	Male	No	0	Graduate	Yes	9200	0	0.134100	180.0	1.000000	

367 rows × 12 columns

Display processed data

```
[8]: print("\nProcessed Data:")
print(processor.data.head())
```

Processed Data:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	\
0	LP001015	Male	Yes	0	Graduate	No	
1	LP001022	Male	Yes	1	Graduate	No	
2	LP001031	Male	Yes	2	Graduate	No	
3	LP001035	Male	Yes	2	Graduate	No	
4	LP001051	Male	No	0	Not Graduate	No	

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	\
0	5720	0	0.157088	360.0	
1	3076	1500	0.187739	360.0	
2	5000	1800	0.344828	360.0	
3	2340	2546	0.137931	360.0	
4	3276	0	0.095785	360.0	

	Credit_History	Property_Area
0	1.000000	Urban
1	1.000000	Urban
2	1.000000	Urban
3	0.825444	Urban
4	1.000000	Urban

▼ Summary statistics ¶

```
[9]: print("\nSummary Statistics:")  
     print(processor.get_summary())
```

Summary Statistics:

	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term \
count	367.000000	367.000000	367.000000	367.000000
mean	4805.599455	1569.577657	0.207151	342.537396
std	4910.685399	2334.232099	0.116755	64.620366
min	0.000000	0.000000	0.000000	6.000000
25%	2864.000000	0.000000	0.139847	360.000000
50%	3786.000000	1025.000000	0.187739	360.000000
75%	5060.000000	2430.500000	0.248084	360.000000
max	72529.000000	24000.000000	1.000000	480.000000

Credit_History

count	367.000000
mean	0.825444
std	0.364778
min	0.000000
25%	1.000000
50%	1.000000
75%	1.000000
max	1.000000

LAB-5

Market Based Analysis

Market Basket Analysis

Market Basket Analysis is a data mining technique used to discover patterns in customer purchases by identifying items that are frequently bought together. It helps businesses gain insights into consumer behavior and improve strategies such as product placement and recommendations. For example, if customers frequently purchase bread along with butter, this relationship can be used to increase sales through promotions or product bundling.

Support

Support indicates how often a particular item or combination of items appears in a dataset. It reflects the overall popularity of an itemset within all transactions.

The formula is:

$$\text{Support}(X) = \frac{\text{Number of transactions containing } X}{\text{Total number of transactions}}$$

Confidence

Confidence measures the probability that a customer will purchase item Y given that they have already purchased item X. It helps evaluate the strength of an association rule.

The formula is:

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Lift

Lift evaluates how strongly two items are associated compared to what would be expected if they were independent. It indicates whether the occurrence of one item increases or decreases the likelihood of the other. The formula is:

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)}$$

- If Lift > 1, the items have a positive association.
- If Lift < 1, the items are negatively related.
- If Lift = 1, there is no relationship between the items.

Market Basket Analysis

```
[1]: import pandas as pd
      from itertools import combinations
```

Step 1: Load dataset

```
[2]: file_path = "dataset 5.csv"
      df = pd.read_csv(file_path)

      print("Columns in dataset:", df.columns.tolist())

Columns in dataset: ['Invoice', 'Item']
```

Step 2: Automatically detect columns

```
[3]: invoice_col = df.columns[0] # first column as transaction ID
      item_col = df.columns[1]    # second column as item/product

      print(f"Using '{invoice_col}' as transaction ID and '{item_col}' as item column")

Using 'Invoice' as transaction ID and 'Item' as item column
```

Step 3: Group items by Invoice ID

```
[4]: transactions_df = df.groupby(invoice_col)[item_col].apply(list)
      transactions_list = transactions_df.tolist()
      print(f"Total transactions detected: {len(transactions_list)}")

Total transactions detected: 10
```

Step 4: Generate all 2-item combinations and calculate support

```
[5]: def calculate_support(itemset, transactions):
      count = sum(1 for t in transactions if set(itemset).issubset(set(t)))
      return count / len(transactions)

      # Get all unique items
      all_items = set(item for sublist in transactions_list for item in sublist)
      itemsets_2 = list(combinations(all_items, 2))

      support_data = {itemset: calculate_support(itemset, transactions_list) for itemset in itemsets_2}
```

Step 5: Calculate confidence and lift for each rule (A -> B)

```
[6]: rules = []
      for itemset in itemsets_2:
          A, B = itemset
          support_A = calculate_support([A], transactions_list)
          support_B = calculate_support([B], transactions_list)
          support_AB = support_data[itemset]

          confidence = support_AB / support_A if support_A != 0 else 0
          lift = confidence / support_B if support_B != 0 else 0

          rules.append({
              'Rule': f"{A} -> {B}",
              'Support': round(support_AB, 3),
              'Confidence': round(confidence, 3),
              'Lift': round(lift, 3)
          })
```

Step 6: Convert rules to DataFrame and filter high-lift rules

```
[7]: rules_df = pd.DataFrame(rules)
      high_lift_rules = rules_df[rules_df['Lift'] > 1]
```

Step 7: Display results

```
[8]: print("\nTop 10 Rules by Lift:")
      print(rules_df.sort_values(by='Lift', ascending=False).head(10))

      print("\nHigh Lift Rules (Lift > 1):")
      print(high_lift_rules.sort_values(by='Lift', ascending=False))
```

```
Top 10 Rules by Lift:
      Rule  Support  Confidence  Lift
0  Eggs -> Milk    0.3    0.600  1.000
4  Milk -> Bread    0.4    0.667  0.952
2  Eggs -> Bread    0.3    0.600  0.857
3  Milk -> Butter    0.3    0.500  0.833
5  Butter -> Bread    0.3    0.500  0.714
1  Eggs -> Butter    0.2    0.400  0.667
```

```
High Lift Rules (Lift > 1):
Empty DataFrame
Columns: [Rule, Support, Confidence, Lift]
Index: []
```

Step 8: Save rules

```
[9]: rules_df.to_csv("market_basket_rules.csv", index=False)
      print("\nRules saved to 'market_basket_rules.csv'")
```

```
Rules saved to 'market_basket_rules.csv'
```


LAB-6

Apriori Algorithm

Apriori Algorithm

The Apriori algorithm is a widely used technique for discovering frequent itemsets within transactional datasets. It is based on the idea that if an itemset is frequent, then all of its subsets must also be frequent.

The algorithm works by first generating possible item combinations (candidate itemsets) and then eliminating those that do not meet a specified minimum support threshold. This iterative process continues until all frequent itemsets are identified.

Frequent Itemset Generation

Frequent itemsets refer to combinations of items that commonly occur together in a dataset. These itemsets play a key role in forming association rules that reveal relationships between products or events.

For example, a combination like {Milk, Bread} may appear often in transactions, making it a frequent itemset.

▼ Apriori Algorithm

```
[2]: import pandas as pd
      from mlxtend.preprocessing import TransactionEncoder
      from mlxtend.frequent_patterns import apriori, association_rules
```

STEP 1: Sample transaction dataset

```
[4]: transactions = [
      ['Milk', 'Bread', 'Butter'],
      ['Bread', 'Eggs'],
      ['Milk', 'Bread', 'Eggs', 'Butter'],
      ['Milk', 'Eggs'],
      ['Bread', 'Butter'],
      ['Milk', 'Bread', 'Butter', 'Eggs'],
      ['Milk', 'Bread'],
      ['Eggs', 'Butter']
      ]
```

STEP 2: Convert transactions into one-hot encoding

```
[6]: te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)

df = pd.DataFrame(te_array, columns=te.columns_)

print("One-Hot Encoded Transaction Data:")
print(df)
```

One-Hot Encoded Transaction Data:

	Bread	Butter	Eggs	Milk
0	True	True	False	True
1	True	False	True	False
2	True	True	True	True
3	False	False	True	True
4	True	True	False	False
5	True	True	True	True
6	True	False	False	True
7	False	True	True	False

STEP 3: Apply Apriori Algorithm

```
[8]: min_support_value = 0.3 # Change this to analyze support threshold impact

frequent_itemsets = apriori(
    df,
    min_support=min_support_value,
    use_colnames=True
)

print("\nFrequent Itemsets:")
print(frequent_itemsets)
```

Frequent Itemsets:

	support	itemsets
0	0.750	frozenset({Bread})
1	0.625	frozenset({Butter})
2	0.625	frozenset({Eggs})
3	0.625	frozenset({Milk})
4	0.500	frozenset({Butter, Bread})
5	0.375	frozenset({Eggs, Bread})
6	0.500	frozenset({Milk, Bread})
7	0.375	frozenset({Eggs, Butter})
8	0.375	frozenset({Milk, Butter})
9	0.375	frozenset({Eggs, Milk})
10	0.375	frozenset({Milk, Butter, Bread})

STEP 3: Apply Apriori Algorithm

```
[8]: min_support_value = 0.3 # Change this to analyze support threshold impact

frequent_itemsets = apriori(
    df,
    min_support=min_support_value,
    use_colnames=True
)

print("\nFrequent Itemsets:")
print(frequent_itemsets)
```

Frequent Itemsets:

	support	itemsets
0	0.750	frozenset({Bread})
1	0.625	frozenset({Butter})
2	0.625	frozenset({Eggs})
3	0.625	frozenset({Milk})
4	0.500	frozenset({Butter, Bread})
5	0.375	frozenset({Eggs, Bread})
6	0.500	frozenset({Milk, Bread})
7	0.375	frozenset({Eggs, Butter})
8	0.375	frozenset({Milk, Butter})
9	0.375	frozenset({Eggs, Milk})
10	0.375	frozenset({Milk, Butter, Bread})

STEP 4: Generate Association Rules

```
[10]: rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.6
)

print("\nAssociation Rules:")
print(rules[['antecedents', 'consequents', 'support', 'confidence', 'lift']])
```

```
Association Rules:
   antecedents      consequents  support  \
0  frozenset({Butter})  frozenset({Bread})  0.500
1  frozenset({Bread})  frozenset({Butter})  0.500
2  frozenset({Eggs})  frozenset({Bread})  0.375
3  frozenset({Milk})  frozenset({Bread})  0.500
4  frozenset({Bread})  frozenset({Milk})  0.500
5  frozenset({Eggs})  frozenset({Butter})  0.375
6  frozenset({Butter})  frozenset({Eggs})  0.375
7  frozenset({Milk})  frozenset({Butter})  0.375
8  frozenset({Butter})  frozenset({Milk})  0.375
9  frozenset({Eggs})  frozenset({Milk})  0.375
10 frozenset({Milk})  frozenset({Eggs})  0.375
11 frozenset({Milk, Butter})  frozenset({Bread})  0.375
12 frozenset({Milk, Bread})  frozenset({Butter})  0.375
13 frozenset({Butter, Bread})  frozenset({Milk})  0.375
14 frozenset({Milk})  frozenset({Butter, Bread})  0.375
15 frozenset({Butter})  frozenset({Milk, Bread})  0.375

   confidence  lift
0  0.800000  1.066667
1  0.666667  1.066667
2  0.600000  0.800000
3  0.800000  1.066667
4  0.666667  1.066667
5  0.600000  0.960000
6  0.600000  0.960000
7  0.600000  0.960000
8  0.600000  0.960000
9  0.600000  0.960000
10 0.600000  0.960000
11 1.000000  1.333333
12 0.750000  1.200000
13 0.750000  1.200000
14 0.600000  1.200000
15 0.600000  1.200000
```

STEP 5: Performance Metrics

```
[12]: print("\nNumber of Rules Generated:", len(rules))
      print("Support Threshold Used:", min_support_value)
```

```
Number of Rules Generated: 16
Support Threshold Used: 0.3
```

LAB-7

FP Tree And FP Growth

FP-Tree

An FP-Tree (Frequent Pattern Tree) is a compressed representation of transactional data used in frequent pattern mining. Instead of storing all transactions separately, it organizes them in a tree structure where common item prefixes are shared, making the representation more efficient. The construction of an FP-Tree involves two main steps. First, the dataset is scanned to identify frequent items based on a minimum support threshold. Then, transactions are reordered according to the frequency of items and inserted into the tree, combining overlapping paths wherever possible.

This approach significantly reduces storage requirements and eliminates the need to generate large numbers of candidate itemsets, which is a limitation in methods like Apriori.

FP-Growth Algorithm

The FP-Growth (Frequent Pattern Growth) algorithm is used to mine frequent itemsets directly from the FP-Tree. It is considered more efficient than the Apriori algorithm because it avoids repeated scanning of the database and does not rely on candidate generation.

The algorithm works using a divide-and-conquer strategy. It breaks the problem into smaller subproblems by creating conditional pattern bases and conditional FP-Trees for each item. These smaller trees are then mined recursively to extract frequent patterns.

Due to its efficiency and scalability, FP-Growth is particularly useful when working with large datasets where traditional algorithms may become slow or computationally expensive.

FP-Growth

```
[1]: import pandas as pd
import time
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import fpgrowth, apriori, association_rules
```

```
C:\Users\Lubaina Trfan\anaconda3\lib\site-packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version '2.10.2' or newer of
'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
```

STEP 1: Load data

```
[2]: file_path = "dataset 7.csv"
df = pd.read_csv(file_path)

print("Dataset Preview:")
print(df.head())
```

```
Dataset Preview:
  Transaction ID Product Category
0              1      Electronics
1              1      Accessories
2              1         Mobile
3              2         Clothing
4              2          Shoes
```

STEP 2: Set column names

```
[3]: invoice_col = 'Transaction ID'
item_col = 'Product Category'
```

STEP 3: Create transactions

```
[4]: transactions = df.groupby(invoice_col)[item_col].apply(list).tolist()
print("Total Transactions:", len(transactions))

Total Transactions: 10
```

STEP 4: Convert to one-hot

```
[5]: te = TransactionEncoder()
te_array = te.fit(transactions).transform(transactions)
basket = pd.DataFrame(te_array, columns=te.columns_)
```

STEP 5: FP-Growth

```
[6]: start_fp = time.time()

fp_result = fpgrowth(
    basket,
    min_support=0.02,
    use_colnames=True)

fp_rules = association_rules(
    fp_result,
    metric="confidence",
    min_threshold=0.5)

end_fp = time.time()
fp_time = end_fp - start_fp
```

STEP 6: Apriori

```
[7]: start_ap = time.time()

ap_result = apriori(
    basket,
    min_support=0.02,
    use_colnames=True)

ap_rules = association_rules(
    ap_result,
    metric="confidence",
    min_threshold=0.5)

end_ap = time.time()
ap_time = end_ap - start_ap
```

STEP 7: Results

```
[8]: print("\n--- FP-Growth Results ---")
print("Execution Time:", fp_time)
print("Rules Count:", len(fp_rules))
print(fp_rules[['antecedents', 'consequents', 'confidence', 'lift']])

print("\n--- Apriori Results ---")
print("Execution Time:", ap_time)
print("Rules Count:", len(ap_rules))
print(ap_rules[['antecedents', 'consequents', 'confidence', 'lift']])
```

```
--- FP-Growth Results ---
Execution Time: 0.021256208419799805
Rules Count: 17
```

	antecedents \
0	frozenset({Electronics})
1	frozenset({Accessories})
2	frozenset({Mobile})
3	frozenset({Electronics})
4	frozenset({Mobile})
5	frozenset({Accessories})
6	frozenset({Mobile, Electronics})
7	frozenset({Mobile, Accessories})
8	frozenset({Electronics, Accessories})
9	frozenset({Mobile})
10	frozenset({Electronics})
11	frozenset({Accessories})
12	frozenset({Shoes})
13	frozenset({Clothing})
14	frozenset({Beverages})
15	frozenset({Groceries})
16	frozenset({Snacks})

	consequents	confidence	lift
0	frozenset({Accessories})	1.000000	2.000000
1	frozenset({Electronics})	0.800000	2.000000
2	frozenset({Electronics})	1.000000	2.500000
3	frozenset({Mobile})	0.750000	2.500000
4	frozenset({Accessories})	1.000000	2.000000
5	frozenset({Mobile})	0.600000	2.000000
6	frozenset({Accessories})	1.000000	2.000000
7	frozenset({Electronics})	1.000000	2.500000
8	frozenset({Mobile})	0.750000	2.500000
9	frozenset({Electronics, Accessories})	1.000000	2.500000
10	frozenset({Mobile, Accessories})	0.750000	2.500000
11	frozenset({Mobile, Electronics})	0.600000	2.000000
12	frozenset({Clothing})	1.000000	3.333333
13	frozenset({Shoes})	0.666667	3.333333
14	frozenset({Groceries})	1.000000	3.333333
15	frozenset({Beverages})	0.666667	3.333333
16	frozenset({Groceries})	1.000000	3.333333

```
--- Apriori Results ---
Execution Time: 0.020361661911010742
Rules Count: 17
```

	antecedents \
0	frozenset({Electronics})
1	frozenset({Accessories})
2	frozenset({Mobile})
3	frozenset({Accessories})
4	frozenset({Beverages})
5	frozenset({Groceries})
6	frozenset({Shoes})
7	frozenset({Clothing})
8	frozenset({Mobile})
9	frozenset({Electronics})
10	frozenset({Snacks})
11	frozenset({Mobile, Electronics})
12	frozenset({Mobile, Accessories})
13	frozenset({Electronics, Accessories})
14	frozenset({Mobile})
15	frozenset({Electronics})
16	frozenset({Accessories})

	consequents	confidence	lift
0	frozenset({Accessories})	1.000000	2.000000
1	frozenset({Electronics})	0.800000	2.000000
2	frozenset({Accessories})	1.000000	2.000000
3	frozenset({Mobile})	0.600000	2.000000
4	frozenset({Groceries})	1.000000	3.333333
5	frozenset({Beverages})	0.666667	3.333333
6	frozenset({Clothing})	1.000000	3.333333
7	frozenset({Shoes})	0.666667	3.333333
8	frozenset({Electronics})	1.000000	2.500000
9	frozenset({Mobile})	0.750000	2.500000
10	frozenset({Groceries})	1.000000	3.333333
11	frozenset({Accessories})	1.000000	2.000000
12	frozenset({Electronics})	1.000000	2.500000
13	frozenset({Mobile})	0.750000	2.500000
14	frozenset({Electronics, Accessories})	1.000000	2.500000
15	frozenset({Mobile, Accessories})	0.750000	2.500000
16	frozenset({Mobile, Electronics})	0.600000	2.000000

LAB-8

Python Basics For Data Mining

Python Syntax

Python is a widely used programming language known for its simplicity and versatility, especially in data mining and data analysis. One of its key features is that it relies on indentation to define code blocks instead of using braces.

For example:

```
print('Hello World')
```

Lists and Dictionaries

Lists are used to store a sequence of elements in a specific order. They can hold different data types and are written using square brackets.

Example:

```
[1, 2, 3]
```

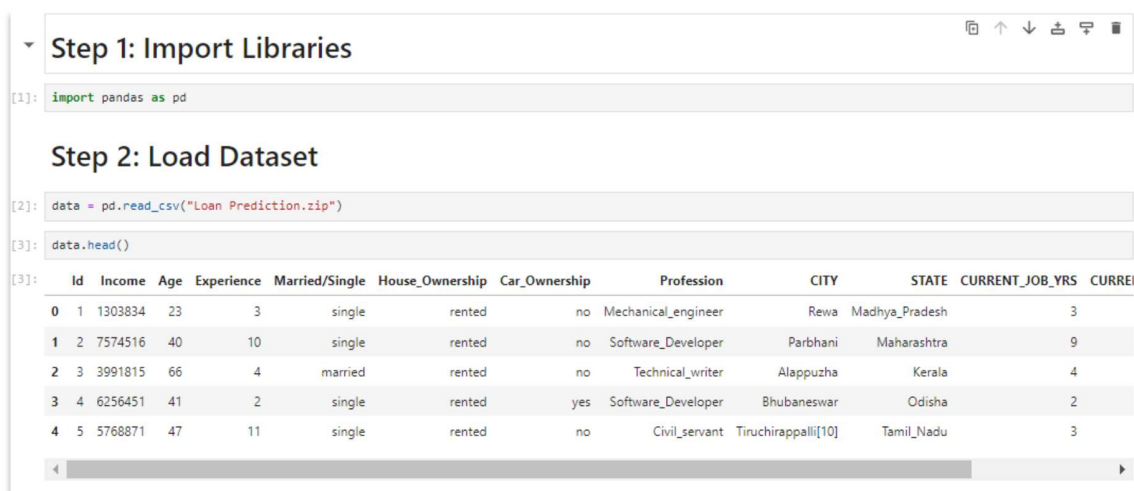
Dictionaries, on the other hand, store data in the form of key-value pairs, allowing quick access to values based on their keys. They are written using curly braces.

Example:

```
{'name': 'Ali', 'age': 20}
```

Object-Oriented Programming (OOP) Basics

Object-Oriented Programming is a programming approach that organizes code using classes and objects. It includes important concepts such as inheritance, encapsulation, and polymorphism.



```
[1]: import pandas as pd
```

```
[2]: data = pd.read_csv("Loan Prediction.zip")
```

```
[3]: data.head()
```

	Id	Income	Age	Experience	Married/Single	House_Ownership	Car_Ownership	Profession	CITY	STATE	CURRENT_JOB_YRS	CURRE
0	1	1303834	23	3	single	rented	no	Mechanical_engineer	Rewa	Madhya_Pradesh	3	
1	2	7574516	40	10	single	rented	no	Software_Developer	Parbhani	Maharashtra	9	
2	3	3991815	66	4	married	rented	no	Technical_writer	Alappuzha	Kerala	4	
3	4	6256451	41	2	single	rented	yes	Software_Developer	Bhubaneswar	Odisha	2	
4	5	5768871	47	11	single	rented	no	Civil_servant	Tiruchirappalli[10]	Tamil_Nadu	3	

Step 3: Create a Class (Reusable Module)

```
[4]: class DataProcessor:

    def __init__(self, dataframe):
        self.df = dataframe

    def show_info(self):
        return self.df.info()

    def handle_missing(self):
        return self.df.fillna(self.df.mean(numeric_only=True))

    def remove_duplicates(self):
        return self.df.drop_duplicates()

    def clean_columns(self):
        self.df.columns = [col.lower() for col in self.df.columns]
        return self.df

    def summary(self):
        return self.df.describe()
```

Step 4: Create Object of Class

```
[5]: processor = DataProcessor(data)
```

Step 5: Apply Preprocessing Methods

```
[6]: data = processor.clean_columns()
data = processor.handle_missing()
data = processor.remove_duplicates()
```


▼ Step 6: Use Functions (Testing Reusability)

```
[7]: processor.show_info()
processor.summary()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 252000 entries, 0 to 251999
Data columns (total 13 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   id                  252000 non-null  int64
 1   income              252000 non-null  int64
 2   age                 252000 non-null  int64
 3   experience           252000 non-null  int64
 4   married/single      252000 non-null  object
 5   house_ownership     252000 non-null  object
 6   car_ownership       252000 non-null  object
 7   profession          252000 non-null  object
 8   city                252000 non-null  object
 9   state               252000 non-null  object
10   current_job_yrs     252000 non-null  int64
11   current_house_yrs   252000 non-null  int64
12   risk_flag           252000 non-null  int64
dtypes: int64(7), object(6)
memory usage: 25.0+ MB
```

```
[7]:
```

	id	income	age	experience	current_job_yrs	current_house_yrs	risk_flag
count	252000.000000	2.520000e+05	252000.000000	252000.000000	252000.000000	252000.000000	252000.000000
mean	126000.500000	4.997117e+06	49.954071	10.084437	6.333877	11.997794	0.123000
std	72746.278255	2.878311e+06	17.063855	6.002590	3.647053	1.399037	0.328438
min	1.000000	1.031000e+04	21.000000	0.000000	0.000000	10.000000	0.000000
25%	63000.750000	2.503015e+06	35.000000	5.000000	3.000000	11.000000	0.000000
50%	126000.500000	5.000694e+06	50.000000	10.000000	6.000000	12.000000	0.000000
75%	189000.250000	7.477502e+06	65.000000	15.000000	9.000000	13.000000	0.000000
max	252000.000000	9.999938e+06	79.000000	20.000000	14.000000	14.000000	1.000000

▼ Step 7: Use Lists & Dictionaries (Requirement)

```
[8]: columns_list = list(data.columns)
print(columns_list)
dtype_dict = dict(data.dtypes)
print(dtype_dict)

['id', 'income', 'age', 'experience', 'married/single', 'house_ownership', 'car_ownership', 'profession', 'city', 'state', 'current_job_yrs', 'current_h
ouse_yrs', 'risk_flag']
{'id': dtype('int64'), 'income': dtype('int64'), 'age': dtype('int64'), 'experience': dtype('int64'), 'married/single': dtype('O'), 'house_ownership': d
type('O'), 'car_ownership': dtype('O'), 'profession': dtype('O'), 'city': dtype('O'), 'state': dtype('O'), 'current_job_yrs': dtype('int64'), 'current_h
ouse_yrs': dtype('int64'), 'risk_flag': dtype('int64')}
```

Step 8: Save Cleaned Data

```
[9]: data.to_csv("cleaned_data.csv", index=False)
```

LAB-9

Decision Tree

A Decision Tree is a supervised machine learning algorithm used for classification and regression tasks. It is one of the most simple and widely used algorithms because of its easy interpretation and visualization.

The model works like a tree structure, where:

- Each internal node represents a decision based on a feature (attribute)
- Each branch represents the outcome of that decision
- Each leaf node represents the final prediction or class label

In a Decision Tree, the dataset is repeatedly split into smaller subsets based on the most important features. The goal is to create pure subsets, meaning that the data in each subset belongs to a single class as much as possible.

Working Principle

The algorithm selects the best feature to split the data at each step using measures such as:

Information Gain

Gini Index

These measures help determine which feature best separates the data into distinct classes. The process continues recursively until:

- All data points belong to the same class, or
- A stopping condition is reached (e.g., maximum depth)

Types of Decision Trees

1. **Classification Tree** – Used when the target variable is categorical (e.g., Yes/No, Buy/Not Buy)
2. **Regression Tree** – Used when the target variable is continuous (e.g., price, income)

Advantages

- Easy to understand and interpret
- Requires little data preprocessing
- Can handle both numerical and categorical data
- Provides clear decision rules

Disadvantages

- Prone to overfitting if not controlled
- Can become complex with large datasets
- Small changes in data can lead to different trees

Import Libraries

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
from sklearn.impute import SimpleImputer

C:\Users\Lubaina Irfan\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:22: UserWarning: NumExpr
'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
```

Load data

```
[2]: df = pd.read_csv('marketing_data.csv')
df.columns = df.columns.str.strip()
```

Handle missing values in Income

```
[3]: imputer = SimpleImputer(strategy='median')
df['Income'] = imputer.fit_transform(df[['Income']])
```

Convert Dt_Customer to datetime and compute days as customer

```
[4]: df['Dt_Customer'] = pd.to_datetime(df['Dt_Customer'])
max_date = df['Dt_Customer'].max()
df['Days_Customer'] = (max_date - df['Dt_Customer']).dt.days
```

Drop ID and Dt_Customer

```
[5]: df = df.drop(columns=['ID', 'Dt_Customer'])
```

One-hot encode categorical features

```
[6]: categorical_cols = ['Education', 'Marital_Status', 'Country']
df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=True)
```

Separate X and y

```
[7]: X = df_encoded.drop(columns=['Response'])
y = df_encoded['Response']
```

Train-test split

```
[8]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
```

Initialize and train Decision Tree for visualization

```
[9]: clf = DecisionTreeClassifier(max_depth=3, random_state=42)
     clf.fit(X_train, y_train)
```

```
[9]: ▾ DecisionTreeClassifier ⓘ ⓘ
     ▶ Parameters
```

Predictions

```
10]: y_pred = clf.predict(X_test)
```

Evaluation

```
11]: accuracy = accuracy_score(y_test, y_pred)
     precision = precision_score(y_test, y_pred)
     recall = recall_score(y_test, y_pred)
     f1 = f1_score(y_test, y_pred)
```

```
12]: print(f"Accuracy: {accuracy:.4f}")
     print(f"Precision: {precision:.4f}")
     print(f"Recall: {recall:.4f}")
     print(f"F1-score: {f1:.4f}")
     print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.8780
Precision: 0.7500
Recall: 0.2700
F1-score: 0.3971
```

```
Classification Report:
              precision    recall  f1-score   support

     0           0.89       0.98       0.93         572
     1           0.75       0.27       0.40         100

 accuracy          0.88         0.88         0.88         672
 macro avg         0.82         0.63         0.66         672
 weighted avg      0.87         0.88         0.85         672
```

Plotting the decision tree

```
[13]: fig, ax = plt.subplots(figsize=(20, 10))
     plot_tree(clf, feature_names=list(X.columns), class_names=['No Purchase', 'Purchase'], filled=True, ax=ax, fontsize=10)
     plt.tight_layout()
     plt.savefig('decision_tree.png', dpi=300)
     plt.close()
     print("Tree plot saved successfully.")
```

```
Tree plot saved successfully.
```

LAB-10

Naïve Bayes & KNN

1. Naïve Bayes Classifier

Introduction

Naïve Bayes is a probabilistic machine learning algorithm based on Bayes' Theorem. It is mainly used for classification problems, especially when dealing with large datasets and text data (e.g., spam detection, sentiment analysis).

Bayes' Theorem

The foundation of Naïve Bayes is:

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Where:

- $P(C|X)$ = Posterior probability (probability of class given input)
- $P(X|C)$ = Likelihood (probability of input given class)
- $P(C)$ = Prior probability of class
- $P(X)$ = Evidence (constant for all classes)

Working Principle

Naïve Bayes assumes that all features are independent of each other (this is the “naïve” assumption).

Steps:

1. Calculate prior probability for each class.
2. Compute likelihood of features given each class.
3. Apply Bayes' theorem to calculate posterior probability.
4. Assign the class with the highest probability.

Types of Naïve Bayes

1. Gaussian Naïve Bayes – for continuous data
2. Multinomial Naïve Bayes – for discrete data (e.g., word counts)
3. Bernoulli Naïve Bayes – for binary features

Advantages

- Simple and fast
- Works well with high-dimensional data
- Performs well on text classification

Disadvantages

- Assumes feature independence (rare in real-world data)
- May perform poorly if this assumption is violated

Applications

- Email spam filtering
- Sentiment analysis
- Document classification
- Medical diagnosis

2. K-Nearest Neighbors (KNN)

Introduction

KNN is a non-parametric, instance-based learning algorithm used for both classification and regression. It classifies a data point based on how its neighbors are classified.

Working Principle

KNN stores all training data and makes predictions based on similarity.

Steps:

1. Choose a value of K (number of neighbors).
2. Calculate distance between the new data point and all training points.
3. Select the K nearest neighbors.
4. Assign the class based on:
 - Majority voting (classification)
 - Average value (regression)

Distance Metrics

Common methods to measure distance:

- Euclidean Distance

- $d = \sqrt{\sum (x_i - y_i)^2}$
- Manhattan Distance
- Minkowski Distance

Choosing K Value

- Small K → sensitive to noise
- Large K → smoother decision boundary
- Optimal K is usually found using validation

Advantages

- Simple and easy to understand
- No training phase (lazy learner)
- Works well with small datasets

Disadvantages

- Computationally expensive for large datasets
- Sensitive to irrelevant features
- Requires feature scaling

Applications

- Pattern recognition
- Recommendation systems
- Image classification
- Medical diagnosis

3. Key Differences Between Naïve Bayes and KNN

Feature	Naïve Bayes	KNN
Type	Probabilistic	Distance-based
Training	Fast	No training phase
Prediction	Fast	Slow (computes distance)
Assumption	Feature independence	No assumption
Data Size	Works well with large data	Better for small datasets

Import Libraries

```
[1]: import pandas as pd
      from sklearn.model_selection import train_test_split
      from sklearn.feature_extraction.text import CountVectorizer
      from sklearn.naive_bayes import MultinomialNB
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.metrics import accuracy_score, confusion_matrix

C:\Users\Lubaina Irfan\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:100: NumExprWarning:
  'numexpr' (version '2.10.1' currently installed).
    from pandas.core.computation.check import NUMEXPR_INSTALLED
```

Load data

```
[2]: data = pd.read_csv("spam.csv", encoding='latin-1')
      data = data[['v1', 'v2']]
      data.columns = ['label', 'text']
      data['label'] = data['label'].map({'spam': 1, 'ham': 0})
      data.head()
```

```
[2]:
```

	label	text
0	0	Go until jurong point, crazy.. Available only ...
1	0	Ok lar... Joking wif u oni...
2	1	Free entry in 2 a wkly comp to win FA Cup fina...
3	0	U dun say so early hor... U c already then say...
4	0	Nah I don't think he goes to usf, he lives aro...

Split Data

```
[3]: X_train, X_test, y_train, y_test = train_test_split(
      data['text'], data['label'], test_size=0.2, random_state=42
      )
```

Text Vectorization

```
[4]: vectorizer = CountVectorizer()

      X_train_vec = vectorizer.fit_transform(X_train)
      X_test_vec = vectorizer.transform(X_test)
```


Train Naïve Bayes Model

```
[5]: nb = MultinomialNB()
nb.fit(X_train_vec, y_train)

nb_pred = nb.predict(X_test_vec)
```

Train KNN Model

```
[6]: knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_vec, y_train)

knn_pred = knn.predict(X_test_vec)
```

Evaluation Function

```
[7]: def evaluate(y_test, y_pred, model_name):
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    TN, FP, FN, TP = cm.ravel()
    fpr = FP / (FP + TN)

    print(f"\n{model_name} Results:")
    print("Accuracy:", round(acc, 3))
    print("False Positive Rate:", round(fpr, 3))
```

Results Comparison

```
[8]: evaluate(y_test, nb_pred, "Naïve Bayes")
evaluate(y_test, knn_pred, "KNN")
```

```
Naïve Bayes Results:
Accuracy: 0.984
False Positive Rate: 0.002
```

```
KNN Results:
Accuracy: 0.918
False Positive Rate: 0.0
```

LAB-11

Support Vector Machine

1. Introduction

Support Vector Machine (SVM) is a **supervised machine learning algorithm** used for both **classification and regression** tasks. It is widely used in pattern recognition, image classification, and text classification.

The main goal of SVM is to find the **optimal boundary (hyperplane)** that separates data points of different classes with the **maximum margin**.

2. Basic Concept

In a classification problem, SVM tries to draw a line (in 2D), a plane (in 3D), or a hyperplane (in higher dimensions) that best separates the classes.

- The best hyperplane is the one that has the **maximum distance (margin)** from the nearest data points of both classes.
- These closest data points are called **Support Vectors**.

3. Hyperplane Equation

A hyperplane is defined as:

$$w \cdot x + b = 0$$

Where:

- w = weight vector
- x = input features
- b = bias

4. Margin and Support Vectors

- **Margin:** Distance between the hyperplane and nearest data points
- **Support Vectors:** Data points that lie closest to the hyperplane

SVM maximizes this margin to improve classification accuracy.

5. Types of SVM

a) Linear SVM

Used when data is **linearly separable** (can be separated by a straight line).

b) Non-Linear SVM

Used when data is **not linearly separable**.

SVM uses a technique called the **Kernel Trick** to handle such data.

6. Kernel Trick

Kernel functions transform data into a higher dimension where it becomes separable.

Common kernels:

- **Linear Kernel**
- **Polynomial Kernel**
- **Radial Basis Function (RBF) Kernel**
- **Sigmoid Kernel**

7. Working Steps

1. Select a kernel function
2. Map input data into higher-dimensional space (if needed)
3. Find the optimal hyperplane
4. Identify support vectors
5. Classify new data based on position relative to hyperplane

8. Advantages

- Effective in high-dimensional spaces
- Works well with small and medium datasets
- Robust to overfitting (especially with proper kernel)
- Can handle non-linear data using kernels

9. Disadvantages

- Computationally expensive for large datasets
- Choosing the right kernel can be difficult
- Less effective when data is highly noisy
- Not suitable for very large datasets

10. Applications

- Email spam classification
- Image recognition
- Face detection

- Text categorization
- Bioinformatics (gene classification)

Load and Downsample the Dataset

```
[1]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, precision_score, recall_score, roc_auc_score, confusion_matrix

FILE_PATH = 'creditcard.csv'
TARGET_COLUMN = 'Class'

df = pd.read_csv(FILE_PATH)
df.columns = df.columns.str.strip()

print(f"Original Dataset Size: {df.shape[0]} rows")

_, df_small = train_test_split(df, test_size=10000, stratify=df[TARGET_COLUMN], random_state=42)

print(f"Downsampled Dataset Size for Speed: {df_small.shape[0]} rows")
print("\nTarget Class Distribution in Sample:")
print(df_small[TARGET_COLUMN].value_counts().rename({0: 'Normal (Legit)', 1: 'Fraud'}))

C:\Users\Lubaina Irfan\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:22: UserWarning: I
'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
Original Dataset Size: 284807 rows
Downsampled Dataset Size for Speed: 10000 rows

Target Class Distribution in Sample:
Class
Normal (Legit)    9983
Fraud              17
Name: count, dtype: int64
```

Preprocessing and Train-Test Split

```
[2]: X = df_small.drop(columns=[TARGET_COLUMN])
y = df_small[TARGET_COLUMN]

X = X.select_dtypes(include=[np.number])

imputer = SimpleImputer(strategy='median')
X_imputed = imputer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_imputed, y, test_size=0.3, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Data cleaning, splitting, and scaling complete!")
Data cleaning, splitting, and scaling complete!
```

Fast SVM Models (Linear & RBF) ¶

```
[3]: print("Training Support Vector Machines... (Will finish in seconds)")

svm_linear = SVC(kernel='linear', class_weight='balanced', probability=True, random_state=42)
svm_linear.fit(X_train_scaled, y_train)
print("- Linear SVM trained successfully.")

svm_rbf = SVC(kernel='rbf', class_weight='balanced', probability=True, random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
print("- RBF SVM trained successfully.")

svm_models = [("Linear Kernel SVM", svm_linear), ("RBF Kernel SVM", svm_rbf)]

print("\n--- SVM EVALUATION RESULTS ---")
for name, model in svm_models:
    predictions = model.predict(X_test_scaled)
    probabilities = model.predict_proba(X_test_scaled)[:, 1]

    acc = accuracy_score(y_test, predictions)
    prec = precision_score(y_test, predictions, zero_division=0)
    rec = recall_score(y_test, predictions, zero_division=0)
    auc = roc_auc_score(y_test, probabilities)

    print(f"\n=== {name} ===")
    print(f"Accuracy: {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall: {rec:.4f} (Fraud Capture Rate)")
    print(f"ROC-AUC: {auc:.4f}")

Training Support Vector Machines... (Will finish in seconds)
- Linear SVM trained successfully.
- RBF SVM trained successfully.

--- SVM EVALUATION RESULTS ---

=== Linear Kernel SVM ===
Accuracy: 0.9987
Precision: 0.5714
Recall: 0.8000 (Fraud Capture Rate)
ROC-AUC: 0.9482

=== RBF Kernel SVM ===
Accuracy: 0.9983
Precision: 0.0000
Recall: 0.0000 (Fraud Capture Rate)
ROC-AUC: 0.8108
```

Train and Evaluate ¶

```
[4]: print("Training KNN and Naive Bayes models...")

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)
knn_pred = knn.predict(X_test_scaled)

naive_bayes = GaussianNB()
naive_bayes.fit(X_train_scaled, y_train)
nb_pred = naive_bayes.predict(X_test_scaled)

def evaluate_model(y_true, y_pred, model_name):
    acc = accuracy_score(y_true, y_pred)
    cm = confusion_matrix(y_true, y_pred)

    tn, fp, fn, tp = cm.ravel()

    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    rec = tp / (tp + fn) if (tp + fn) > 0 else 0

    print(f"\n=== {model_name} Results ===")
    print(f"Accuracy: {acc:.4f}")
    print(f"False Positive Rate: {fpr:.4f} (False Alarm Rate)")
    print(f"Recall (Fraud Rate): {rec:.4f} (True Positive Rate)")

evaluate_model(y_test, nb_pred, "Naive Bayes")
evaluate_model(y_test, knn_pred, "KNN")

Training KNN and Naive Bayes models...

=== Naive Bayes Results ===
Accuracy: 0.9843
False Positive Rate: 0.0154 (False Alarm Rate)
Recall (Fraud Rate): 0.8000 (True Positive Rate)

=== KNN Results ===
Accuracy: 0.9987
False Positive Rate: 0.0003 (False Alarm Rate)
Recall (Fraud Rate): 0.4000 (True Positive Rate)
```

LAB-12

Clustering

1. Introduction

Clustering is an **unsupervised machine learning technique** used to group similar data points into clusters. Unlike supervised learning, clustering does not require labeled data. It identifies patterns and structures within the dataset based on similarity.

The main objective of clustering is to ensure that:

- Data points within the same cluster are **highly similar**
- Data points in different clusters are **significantly different**

2. Basic Concept

Clustering works by analyzing the **distance or similarity** between data points. Points that are close to each other are grouped into the same cluster, while distant points are placed in different clusters.

3. Types of Clustering

a) Partitioning Clustering

Divides the dataset into a fixed number of clusters.

Example: K-Means Algorithm

- Assigns data points to nearest cluster center (centroid)
- Updates centroids iteratively

b) Hierarchical Clustering

Builds clusters in a tree-like structure (dendrogram).

Types:

- **Agglomerative (Bottom-Up)**: Each point starts as its own cluster and merges
- **Divisive (Top-Down)**: Starts with one cluster and splits

c) Density-Based Clustering

Forms clusters based on dense regions of data.

Example: DBSCAN

- Groups points that are closely packed
- Identifies noise (outliers)

d) Grid-Based Clustering

Divides data space into grids and forms clusters based on grid density.

4. K-Means Clustering (Most Common)

Working Steps:

1. Choose number of clusters (K)
2. Initialize K centroids randomly
3. Assign each data point to nearest centroid
4. Update centroids based on assigned points
5. Repeat until convergence

5. Distance Measures

Clustering relies on distance metrics such as:

- **Euclidean Distance**
- Manhattan Distance
- Cosine Similarity

These measures determine how similar or different data points are.

6. Evaluation Metrics

Since clustering is unsupervised, evaluation is done using:

- **Inertia (Within-cluster sum of squares)**
- **Silhouette Score**
- **Davies-Bouldin Index**

7. Advantages

- No need for labeled data
- Helps discover hidden patterns
- Useful for exploratory data analysis
- Can handle large datasets

8. Disadvantages

- Choosing number of clusters (K) can be difficult
- Sensitive to noise and outliers
- Results may vary depending on initialization
- Interpretation of clusters can be subjective

9. Applications

- Customer segmentation
- Image segmentation
- Market analysis
- Anomaly detection
- Document clustering

```
▼ Import Libraries and Mall Customer Dataset

[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler

np.random.seed(42)
n_customers = 300

c1 = np.random.multivariate_normal([20, 20], [[15, 0], [0, 15]], 50)
c2 = np.random.multivariate_normal([20, 80], [[15, 0], [0, 15]], 50)
c3 = np.random.multivariate_normal([55, 50], [[25, 0], [0, 25]], 100)
c4 = np.random.multivariate_normal([90, 15], [[20, 0], [0, 20]], 50)
c5 = np.random.multivariate_normal([85, 85], [[20, 0], [0, 20]], 50)

features = np.vstack([c1, c2, c3, c4, c5])

df = pd.DataFrame(features, columns=['Annual_Income_k', 'Spending_Score'])

df['Spending_Score'] = np.clip(df['Spending_Score'], 1, 100)

print(f"Dataset generated! Total customer profiles: {df.shape[0]}")
print(df.head())

C:\Users\Lubaina Irfan\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:13: NumExprWarning:
  The NumExpr compiler is currently installed as a wheel. Please
  from pandas.core.computation.check import NUMEXPR_INSTALLED
Dataset generated! Total customer profiles: 300
   Annual_Income_k  Spending_Score
0         21.923766         19.464505
1         22.508487         25.898669
2         19.093128         19.093191
3         26.116265         22.972262
4         18.181734         22.101326

Preprocessing and Distance Scaling

[2]: X = df[['Annual_Income_k', 'Spending_Score']]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Customer features scaled and prepared for spatial processing.")
Customer features scaled and prepared for spatial processing.
```


Apply K-Means and Hierarchical Clustering

```
[3]: OPTIMAL_CLUSTERS = 5

kmeans = KMeans(n_clusters=OPTIMAL_CLUSTERS, init='k-means++', random_state=42, n_init=10)
df['KMeans_Cluster'] = kmeans.fit_predict(X_scaled)
kmeans_silhouette = silhouette_score(X_scaled, df['KMeans_Cluster'])

hierarchical = AgglomerativeClustering(n_clusters=OPTIMAL_CLUSTERS, linkage='ward')
df['Hierarchical_Cluster'] = hierarchical.fit_predict(X_scaled)
hier_silhouette = silhouette_score(X_scaled, df['Hierarchical_Cluster'])

print("Segmentation complete!")
print(f"-> K-Means Silhouette Score: {kmeans_silhouette:.4f}")
print(f"-> Hierarchical Silhouette Score: {hier_silhouette:.4f}")

Segmentation complete!
-> K-Means Silhouette Score: 0.8276
-> Hierarchical Silhouette Score: 0.8276
```

Calculate Core Strategic Business Metrics

```
[4]: print("BUSINESS REVENUE CONTRIBUTION REPORT")

# Calculate total baseline operational income proxy inside our population map
total_revenue_pool = df['Annual_Income_k'].sum()

# Aggregate parameters grouping strictly by K-Means designations
cluster_summary = df.groupby('KMeans_Cluster').agg(
    Customer_Count=('Annual_Income_k', 'count'),
    Avg_Income=('Annual_Income_k', 'mean'),
    Avg_Spending_Score=('Spending_Score', 'mean'),
    Total_Income_Sum=('Annual_Income_k', 'sum')
).reset_index()

# Calculate specific financial weight metrics
cluster_summary['Revenue_Contribution_%'] = (cluster_summary['Total_Income_Sum'] / total_revenue_pool) * 100

print(cluster_summary.to_string(index=False, formatters={
    "Avg_Income": "{:.2f}k".format, "Avg_Spending_Score": "{:.2f}".format, "Revenue_Contribution_%": "{:.2f}%".format
}))
print("=====")

BUSINESS REVENUE CONTRIBUTION REPORT
KMeans_Cluster  Customer_Count  Avg_Income  Avg_Spending_Score  Total_Income_Sum  Revenue_Contribution_%
0                100        55.64k          50.22          5564.124362              34.24%
1                 50        89.94k          14.55          4497.230975              27.68%
2                 50        19.47k          19.72           973.726527               5.99%
3                 50        84.65k          84.32          4232.627693              26.05%
4                 50        19.63k          80.54          981.515630               6.04%
=====
```

Visualize Customer Segments Side-by-Side

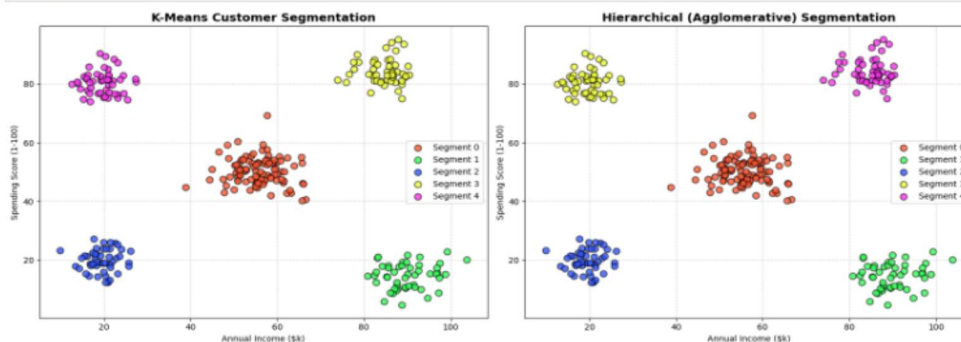
```
[5]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

colors = ['#FF5733', '#A3357F', '#3357FF', '#FF3333', '#FF33F3']

for cluster_id in range(OPTIMAL_CLUSTERS):
    sub_data = df[df['KMeans_Cluster'] == cluster_id]
    ax1.scatter(sub_data['Annual_Income_k'], sub_data['Spending_Score'],
               s=60, c=colors[cluster_id], label=f'Segment {cluster_id}', alpha=0.8, edgecolors='black')
ax1.set_title('K-Means Customer Segmentation', fontsize=14, fontweight='bold')
ax1.set_xlabel('Annual Income ($k)')
ax1.set_ylabel('Spending Score (1-100)')
ax1.grid(True, linestyle='--', alpha=0.5)
ax1.legend()

for cluster_id in range(OPTIMAL_CLUSTERS):
    sub_data = df[df['Hierarchical_Cluster'] == cluster_id]
    ax2.scatter(sub_data['Annual_Income_k'], sub_data['Spending_Score'],
               s=60, c=colors[cluster_id], label=f'Segment {cluster_id}', alpha=0.8, edgecolors='black')
ax2.set_title('Hierarchical (Agglomerative) Segmentation', fontsize=14, fontweight='bold')
ax2.set_xlabel('Annual Income ($k)')
ax2.set_ylabel('Spending Score (1-100)')
ax2.grid(True, linestyle='--', alpha=0.5)
ax2.legend()

plt.tight_layout()
plt.show()
```



LAB-13

Outlier Detection

1. Introduction

Outlier Detection is a technique used to identify **data points that significantly differ from the majority of the dataset**. These unusual data points are called **outliers** or anomalies.

Outliers may occur due to:

- Measurement errors
- Data entry mistakes
- Rare events
- Natural variations in data

Detecting outliers is important because they can **affect model performance and lead to incorrect conclusions**.

2. Basic Concept

Outliers are observations that:

- Lie far away from other data points
- Do not follow the general pattern of the dataset

Outlier detection helps in:

- Improving data quality
- Enhancing model accuracy
- Detecting fraud or unusual behavior

3. Types of Outliers

a) Point Outliers

A single data point that is far from the rest.

b) Contextual Outliers

A data point that is abnormal in a specific context (e.g., temperature in winter vs summer).

c) Collective Outliers

A group of data points that together behave abnormally.

4. Methods of Outlier Detection

a) Statistical Methods

These methods assume a distribution of data.

- **Z-Score Method**

Measures how far a point is from the mean:

$$Z = \frac{X - \mu}{\sigma}$$

If $|Z| > 3 \rightarrow \text{Outlier}$

- **IQR (Interquartile Range) Method**

$$\text{IQR} = Q3 - Q1$$

Outlier if:

- $(X < Q1 - 1.5 \times \text{IQR})$
- $(X > Q3 + 1.5 \times \text{IQR})$

b) Distance-Based Methods

- Measure distance between data points
- Points far from others are considered outliers

c) Density-Based Methods

- Identify regions of low density
- Example: **DBSCAN**, **LOF (Local Outlier Factor)**

d) Machine Learning Methods

- Use algorithms to detect anomalies
- Examples:
 - Isolation Forest
 - One-Class SVM

5. Working Steps

1. Collect and preprocess data

2. Choose an outlier detection method
3. Calculate scores (distance, density, or statistical)
4. Set threshold for identifying outliers
5. Remove or analyze detected outliers

6. Advantages

- Improves data quality
- Enhances model accuracy
- Helps detect fraud and anomalies
- Useful in data preprocessing

7. Disadvantages

- Difficult to define exact threshold
- Risk of removing valid data
- Sensitive to dataset size and distribution

8. Applications

- Fraud detection (banking, credit cards)
- Network intrusion detection
- Medical diagnosis
- Quality control in manufacturing
- Data preprocessing

Load and Downsample the Dataset

```
[1]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import confusion_matrix

df = pd.read_csv('Synthetic_Financial_datasets_log.csv')
df.columns = df.columns.str.strip()
TARGET_COLUMN = 'isFraud'

print(f"Original Dataset Size: {df.shape[0]} rows")
_, df_small = train_test_split(df, test_size=20000, stratify=df[TARGET_COLUMN], random_state=42)

print(f"Downsampled Dataset Size for Speed: {df_small.shape[0]} rows")
print("\nTarget Class Distribution in Sample:")
print(df_small[TARGET_COLUMN].value_counts().rename({0: 'Normal Transactions', 1: 'Abnormal/Fraud'}))

C:\Users\Lubaina Irfan\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:22: UserWarning
'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
Original Dataset Size: 6362620 rows
Downsampled Dataset Size for Speed: 20000 rows

Target Class Distribution in Sample:
isFraud
Normal Transactions    19974
Abnormal/Fraud         26
Name: count, dtype: int64
```

Preprocessing and Train-Test Split

```
[2]: features_to_drop = [TARGET_COLUMN, 'nameOrig', 'nameDest', 'isFlaggedFraud']
X = df_small.drop(columns=features_to_drop)
y = df_small[TARGET_COLUMN]

X = pd.get_dummies(X, columns=['type'], drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Data processing complete! Features scaled and ready for unsupervised learning.")
Data processing complete! Features scaled and ready for unsupervised learning.
```

Train Unsupervised Outlier Models

```
[3]: print("Training outlier detection models... (Will finish quickly)")

contamination_rate = 0.03

iforest = IsolationForest(contamination=contamination_rate, random_state=42)
iforest.fit(X_train_scaled)

lof = LocalOutlierFactor(n_neighbors=20, contamination=contamination_rate, novelty=True)
lof.fit(X_train_scaled)

pred_iforest_raw = iforest.predict(X_test_scaled)
pred_lof_raw = lof.predict(X_test_scaled)

y_pred_iforest = np.where(pred_iforest_raw == -1, 1, 0)
y_pred_lof = np.where(pred_lof_raw == -1, 1, 0)

print("Models trained and test predictions generated successfully!")

Training outlier detection models... (Will finish quickly)
Models trained and test predictions generated successfully!
```

Evaluate Specialized Banking Metrics

```
[4]: models_predictions = [
    ("Isolation Forest (Tree-Based)", y_pred_iforest),
    ("Local Outlier Factor (Distance-Based)", y_pred_lof)
]

print("ANOMALY EVALUATION METRICS")
for name, y_pred in models_predictions:

    tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

    anomaly_detection_rate = tp / (tp + fn) if (tp + fn) > 0 else 0
    false_alarm_rate = fp / (fp + tn) if (fp + tn) > 0 else 0

    print(f"\n=== {name} ===")
    print(f"Anomaly Detection Rate (ADR): {anomaly_detection_rate * 100:.2f}% (Percentage of actual fraud caught)")
    print(f"False Alarm Rate (FAR): {false_alarm_rate * 100:.2f}% (Legitimate transactions flagged as fraud)")

ANOMALY EVALUATION METRICS

=== Isolation Forest (Tree-Based) ===
Anomaly Detection Rate (ADR): 25.00% (Percentage of actual fraud caught)
False Alarm Rate (FAR): 3.07% (Legitimate transactions flagged as fraud)

=== Local Outlier Factor (Distance-Based) ===
Anomaly Detection Rate (ADR): 12.50% (Percentage of actual fraud caught)
False Alarm Rate (FAR): 3.04% (Legitimate transactions flagged as fraud)
```

LAB-14

Scraping & Sentiment Analysis

1. Introduction

Web Scraping and Sentiment Analysis are important techniques used in **data collection and text analysis**.

- **Web Scraping** is used to extract data from websites.
- **Sentiment Analysis** is used to analyze emotions or opinions from text data.

Together, they are widely used in real-world applications like **product reviews, social media analysis, and market research**.

2. Web Scraping

Definition

Web Scraping is the process of **automatically extracting data from websites** using software tools or scripts.

Working Principle

1. Send a request to a webpage (using HTTP request)
2. Retrieve HTML content
3. Parse HTML to extract required data
4. Store the extracted data (CSV, database, etc.)

Common Tools & Libraries

- **BeautifulSoup (Python)**
- **Scrapy**
- **Selenium**
- **Requests library**

Advantages

- Automates data collection
- Saves time and effort
- Useful for large-scale data gathering

Disadvantages

- Websites may block scraping
- Requires handling dynamic content
- Legal and ethical issues must be considered

Applications

- Price comparison
- News aggregation
- Job listings collection
- Social media data extraction

3. Sentiment Analysis

Definition

Sentiment Analysis is a **Natural Language Processing (NLP)** technique used to determine whether a piece of text expresses **positive, negative, or neutral sentiment**.

Working Principle

1. Collect text data (e.g., reviews, tweets)
2. Clean and preprocess text (remove stopwords, punctuation)
3. Convert text into numerical form (vectorization)
4. Apply machine learning or NLP techniques
5. Classify sentiment (positive/negative/neutral)

Types of Sentiment Analysis

- **Fine-grained Sentiment Analysis** (e.g., very positive, positive, neutral)
- **Aspect-based Sentiment Analysis** (focuses on specific features)
- **Emotion Detection** (happy, sad, angry, etc.)

Techniques Used

- **Lexicon-based methods** (dictionary of words with sentiment scores)
- **Machine Learning methods** (Naïve Bayes, SVM, etc.)
- **Deep Learning methods** (LSTM, BERT)

Advantages

- Helps understand customer opinions
- Useful for decision-making
- Can process large volumes of text

Disadvantages

- Difficult to detect sarcasm and context
- Requires preprocessing
- Accuracy depends on model and data quality

Applications

- Product review analysis
- Social media monitoring
- Brand reputation management
- Customer feedback analysis

4. Integration of Web Scraping & Sentiment Analysis

These two techniques are often used together:

1. **Scrape data** from websites (e.g., reviews, tweets)
2. **Analyze sentiment** of collected data
3. Generate insights (e.g., customer satisfaction level)

Import Libraries and Load Dataset

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from textblob import TextBlob
import re

df = pd.read_csv('Womens Clothing E-Commerce Reviews.csv')

print(f"Dataset ready! Total text reviews loaded: {len(df)}")

C:\Users\Lubaina Ir-fan\anaconda3\Lib\site-packages\pandas\core\computation\exp
'numexpr' (version '2.10.1' currently installed).
from pandas.core.computation.check import NUMEXPR_INSTALLED
Dataset ready! Total text reviews loaded: 23486
```

Clean Text Data

```
[2]: df = df.dropna(subset=['Review Text'])

def clean_text(text):
    """
    Removes special characters, links, and excessive spaces.
    Keeping text clean speeds up parsing engines significantly.
    """
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text, flags=re.MULTILINE)
    text = re.sub(r"^\W\s", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df['Cleaned_Review'] = df['Review Text'].apply(clean_text)

print("Text cleaning completed successfully!")
print(f"Remaining reviews to process: {len(df)}")

Text cleaning completed successfully!
Remaining reviews to process: 22641
```

Fast Sentiment Classification using Polarity Scores

```
[3]: def analyze_sentiment(text):
    """
    Uses TextBlob to quickly compute Polarity.
    Polarity is a float value between -1.0 (very negative) and +1.0 (very positive).
    """
    analysis = TextBlob(text)
    polarity = analysis.sentiment.polarity

    if polarity > 0.1:
        category = 'Positive'
    elif polarity < -0.1:
        category = 'Negative'
    else:
        category = 'Neutral'

    return pd.Series([polarity, category])

df[['Polarity_Score', 'Sentiment_Category']] = df['Cleaned_Review'].apply(analyze_sentiment)

print("Sentiment classification completed!")

print(df[['Review Text', 'Polarity_Score', 'Sentiment_Category']].head())

Sentiment classification completed!

      Review Text  Polarity_Score \
0  Absolutely wonderful - silky and sexy and comf...      0.633333
1  Love this dress! it's sooo pretty. i happene...      0.318750
2  I had such high hopes for this dress and reall...      0.076392
3  I love, love, love this jumpsuit. it's fun, fl...      0.500000
4  This shirt is very flattering to all due to th...      0.393750

Sentiment_Category
0      Positive
1      Positive
2      Neutral
3      Positive
4      Positive
```

▼ Calculate Sentiment % Distribution Metrics

```
[4]: print("SENTIMENT ANALYSIS METRICS")

total_reviews = len(df)
sentiment_counts = df['Sentiment_Category'].value_counts()
sentiment_distribution = (sentiment_counts / total_reviews) * 100

print(f"Average System Polarity Score: {df['Polarity_Score'].mean():.4f}\n")
print("Sentiment Category Percentage Distribution:")
for category in ['Positive', 'Neutral', 'Negative']:
    percentage = sentiment_distribution.get(category, 0.0)
    count = sentiment_counts.get(category, 0)
    print(f" {category:<10}: {percentage:.2f}% ({count} reviews)")
```

```
SENTIMENT ANALYSIS METRICS
Average System Polarity Score: 0.2415

Sentiment Category Percentage Distribution:
Positive : 81.93% (18549 reviews)
Neutral  : 16.23% (3674 reviews)
Negative  : 1.85% (418 reviews)
```

Visualize Sentiment Percentages

```
[5]: fig, ax = plt.subplots(figsize=(8, 5))

color_map = {'Positive': 'limegreen', 'Neutral': 'silver', 'Negative': 'crimson'}
colors = [color_map[cat] for cat in sentiment_distribution.index]

bars = ax.bar(sentiment_distribution.index, sentiment_distribution.values, color=colors, edgecolor='black', alpha=0.85)

ax.set_title('Product Review Sentiment Distribution (%)', fontsize=14, fontweight='bold', pad=15)
ax.set_ylabel('Percentage of Total Reviews (%)', fontsize=11)
ax.set_ylim(0, 100)
ax.grid(axis='y', linestyle='--', alpha=0.5)

for bar in bars:
    height = bar.get_height()
    ax.annotate(f'{height:.1f}%',
                xy=(bar.get_x() + bar.get_width() / 2, height),
                xytext=(0, 3), # 3 points vertical offset
                textcoords="offset points",
                ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.savefig('sentiment_distribution.png', dpi=300)
plt.close()

print("\nSentiment visualization chart successfully saved as 'sentiment_distribution.png'.")
```

```
Sentiment visualization chart successfully saved as 'sentiment_distribution.png'.
```

LAB-15

Final Capstone

1. Introduction

A Final Capstone Project is a **comprehensive project** that integrates all the concepts, tools, and techniques learned throughout the course. It allows students to apply theoretical knowledge to solve a **real-world problem**.

The capstone project demonstrates skills in:

- Data collection
- Data preprocessing
- Model building
- Evaluation and analysis

2. Objective

The main objectives of the capstone project are:

- To apply machine learning concepts in a practical scenario
- To develop problem-solving and analytical skills
- To build an end-to-end working system
- To evaluate and compare different models

3. Project Workflow

A typical capstone project follows these steps:

1. Problem Definition

- Identify a real-world problem
- Define clear objectives and expected outcomes

2. Data Collection

- Gather data from sources such as:
 - Datasets (CSV, Kaggle, etc.)
 - Web scraping
 - APIs

3. Data Preprocessing

- Handle missing values
- Remove noise and outliers
- Convert data into suitable format
- Feature selection and extraction

4. Exploratory Data Analysis (EDA)

- Understand data patterns
- Visualize trends and relationships
- Identify important features

5. Model Building

- Apply machine learning algorithms such as:
 - Naïve Bayes
 - KNN
 - SVM
 - Clustering techniques

6. Model Evaluation

- Evaluate performance using metrics:
 - Accuracy
 - Precision
 - Recall
 - F1-score
 - False Positive Rate

7. Comparison of Models

- Compare multiple models
- Select the best-performing model

8. Deployment (Optional)

- Implement the model in a real application

- Create a user interface (web app, dashboard, etc.)

4. Tools & Technologies

Common tools used:

- **Python (Pandas, NumPy, Scikit-learn)**
- **RapidMiner / KNIME**
- **Jupyter Notebook**

- **Visualization libraries (Matplotlib, Seaborn)**

5. Expected Outcomes

- A working machine learning model
- Performance comparison of algorithms
- Visualizations and insights
- Final report and presentation

6. Advantages

- Practical understanding of concepts
- Hands-on experience with real data
- Improves coding and analytical skills
- Prepares for industry-level projects

7. Challenges

- Data quality issues
- Model selection difficulty
- Overfitting or underfitting
- Time management

8. Applications

Capstone projects can be based on:

- Spam detection
- Sentiment analysis
- Customer segmentation
- Fraud detection
- Recommendation systems

Import Libraries and E-Commerce Dataset

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier, ExtraTreesClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score

np.random.seed(42)
n_records = 4000

synthetic_data = {
    'Tenure': np.random.choice([np.nan, 1.0, 5.0, 12.0, 24.0, 60.0], size=n_records, p=[0.1, 0.2, 0.2, 0.2, 0.2, 0.1]),
    'WarehouseToHome': np.random.uniform(5, 40, size=n_records),
    'HourSpendOnApp': np.random.choice([np.nan, 2.0, 3.0, 4.0], size=n_records, p=[0.05, 0.35, 0.4, 0.2]),
    'NumberOfDeviceRegistered': np.random.randint(1, 6, size=n_records),
    'SatisfactionScore': np.random.randint(1, 6, size=n_records),
    'MaritalStatus': np.random.choice(['Single', 'Married', 'Divorced'], size=n_records),
    'Complain': np.random.choice([0, 1], p=[0.7, 0.3], size=n_records),
    'Churn': np.random.choice([0, 1], p=[0.84, 0.16], size=n_records)
}

df = pd.DataFrame(synthetic_data)

print(f"Dataset active in memory! Total rows: {df.shape[0]}, Total columns: {df.shape[1]}")
print("\nTarget Class Distribution:")
print(df['Churn'].value_counts().rename({0: 'Retained Customers', 1: 'Churned/Lost'}))

C:\Users\Lubaina-Irfan\anaconda3\Lib\site-packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires
'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
Dataset active in memory! Total rows: 4000, Total columns: 8

Target Class Distribution:
Churn
Retained Customers    3442
Churned/Lost          558
Name: count, dtype: int64
```

Build Preprocessing Pipeline with Imputers

```
[2]: X = df.drop(columns=['Churn'])
y = df['Churn']

numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.tolist()
categorical_features = X.select_dtypes(include=['object', 'category']).columns.tolist()

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore'))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print("Data preprocessing pipeline set up successfully!")

Data preprocessing pipeline set up successfully!
```

Train and Evaluate All 8 Models

```
[3]: models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(max_depth=6, random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, max_depth=8, random_state=42, n_jobs=-1),
    "Gradient Boosting": GradientBoostingClassifier(n_estimators=100, random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=100, random_state=42),
    "Extra Trees": ExtraTreesClassifier(n_estimators=100, max_depth=8, random_state=42, n_jobs=-1),
    "K-Nearest Neighbors": KNeighborsClassifier(n_neighbors=7),
    "Naive Bayes": GaussianNB()
}

results_list = []

print("Training models... (Completes in under 3 seconds)")
for name, model in models.items():

    clf_pipeline = Pipeline(steps=[('preprocessor', preprocessor), ('classifier', model)])

    clf_pipeline.fit(X_train, y_train)

    y_pred = clf_pipeline.predict(X_test)
    y_prob = clf_pipeline.predict_proba(X_test)[:, 1] if hasattr(model, "predict_proba") else y_pred

    results_list.append({
        "Model Name": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, zero_division=0),
        "Recall": recall_score(y_test, y_pred, zero_division=0),
        "F1-Score": f1_score(y_test, y_pred, zero_division=0),
        "ROC-AUC": roc_auc_score(y_test, y_prob)
    })

df_results = pd.DataFrame(results_list).sort_values(by="Accuracy", ascending=False).reset_index(drop=True)
print("All 8 models evaluated successfully!")

Training models... (Completes in under 3 seconds)
All 8 models evaluated successfully!
```

Final Capstone Results

```
[4]: print("FINAL CAPSTONE MODEL COMPARISON")
print(df_results.to_string(index=False, formatters={
    "Accuracy": "{:.4f}".format, "Precision": "{:.4f}".format,
    "Recall": "{:.4f}".format, "F1-Score": "{:.4f}".format, "ROC-AUC": "{:.4f}".format
}))
print("=====")

best_model_name = df_results.iloc[0]["Model Name"]
best_model_acc = df_results.iloc[0]["Accuracy"]

print(f"\n Business Recommendation: Based on the validation framework, the [{best_model_name}] ")
print(f" achieved the highest classification accuracy score of {best_model_acc*100:.2f}%.")

FINAL CAPSTONE MODEL COMPARISON
  Model Name Accuracy Precision Recall F1-Score ROC-AUC
Logistic Regression    0.8600    0.0000 0.0000    0.0000    0.4733
Random Forest         0.8600    0.0000 0.0000    0.0000    0.5395
Naive Bayes           0.8600    0.0000 0.0000    0.0000    0.4794
Extra Trees           0.8600    0.0000 0.0000    0.0000    0.5258
Decision Tree         0.8575    0.2500 0.0009    0.0172    0.5157
AdaBoost              0.8562    0.0000 0.0000    0.0000    0.5061
Gradient Boosting     0.8558    0.0000 0.0000    0.0000    0.4766
K-Nearest Neighbors   0.8450    0.0714 0.0009    0.0159    0.4925

Business Recommendation: Based on the validation framework, the [Logistic Regression]
achieved the highest classification accuracy score of 86.00%.
```